VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY

INTERNATIONAL UNIVERSITY

WEB APPLICATION DEVELOPMENT PROJECT
RAILWAY MANAGEMENT SYSTEM

BY:

TRẦN PHÚC QUÝ - Student ID: ITITWE23047

TRẦN HOÀNG MINH - Student ID: ITCSIU23025

Advisor: Dr. Nguyen Van Sinh

A report submitted to the School of Computer Science and Engineering

in partial fulfillment of the requirements for the Final Project

in Web Application Development course – Semester 1 2025-2026

Ho Chi Minh city, Viet Nam, 2025

Table of Contents

I. INTRODUCTION

This section provides an overview of our Railway Management System project. The system is designed to help users search for trains, book tickets, select coaches and seats, and manage their bookings efficiently. It highlights the main goals of the project, the core functionalities implemented, and briefly touches on some of the technical and design challenges we encountered during development.

1. ABOUT US

The Railway Management System is a web application project developed by a small team of passionate IT students. As sophomores, we are actively building up our experience and skills, aiming toward future internships in the field. This project is part of our journey to explore real-world web development practices and deliver a practical, user-friendly platform. We focus on creating solutions that are not only functional but also intuitive and engaging, with an emphasis on modern design and user experience.

2. ABOUT THE PROJECT

In today’s fast-paced world, booking train tickets can be a time-consuming and frustrating process. Users often struggle with finding available trains, selecting the right coach and seat, and managing their bookings efficiently. The abundance of information and complex booking processes can make travel planning overwhelming.

To tackle this problem, we developed the **Railway Management System** – a web-based application designed to help users easily search for trains, view available coaches and seats, and book tickets seamlessly. The system allows users to search for trains based on source and destination stations, view detailed train information including departure and arrival times, select specific coaches and seat numbers, and manage their bookings. Additionally, the platform supports user authentication and registration to provide a personalized experience. By simplifying the ticket booking process, the Railway Management System aims to make train travel more accessible and convenient for everyone.

3. WORK BREAKDOWN STRUCTURE

The structure tailored for our project can be expressed in Figure 1:

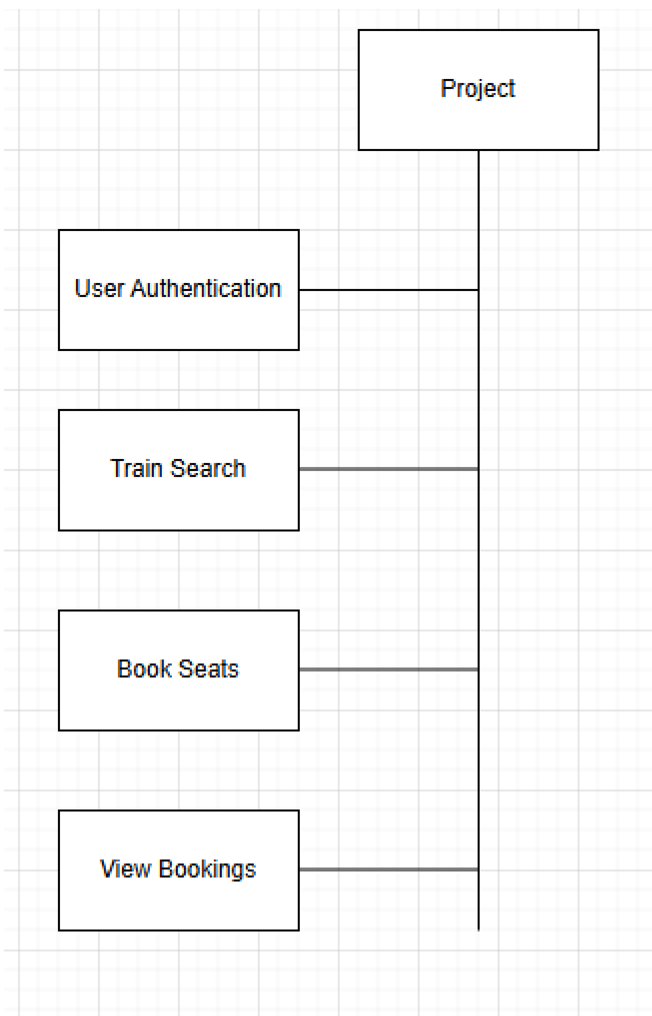


Figure 1: Work Breakdown Structure

3.1 The tasks for User Authentication are shown in Figure 2:

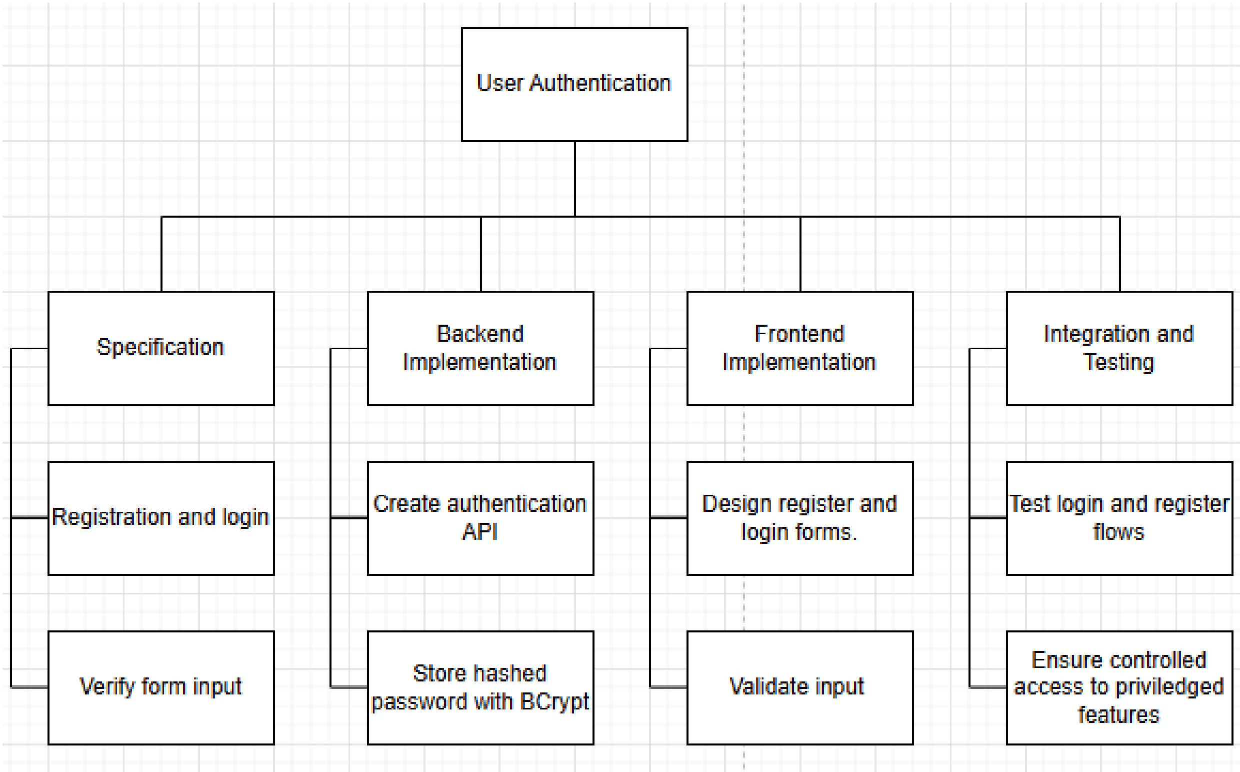


Figure 2: User Authentication Tasks

3.2 The tasks for Train Search & Display are shown in Figure 3:

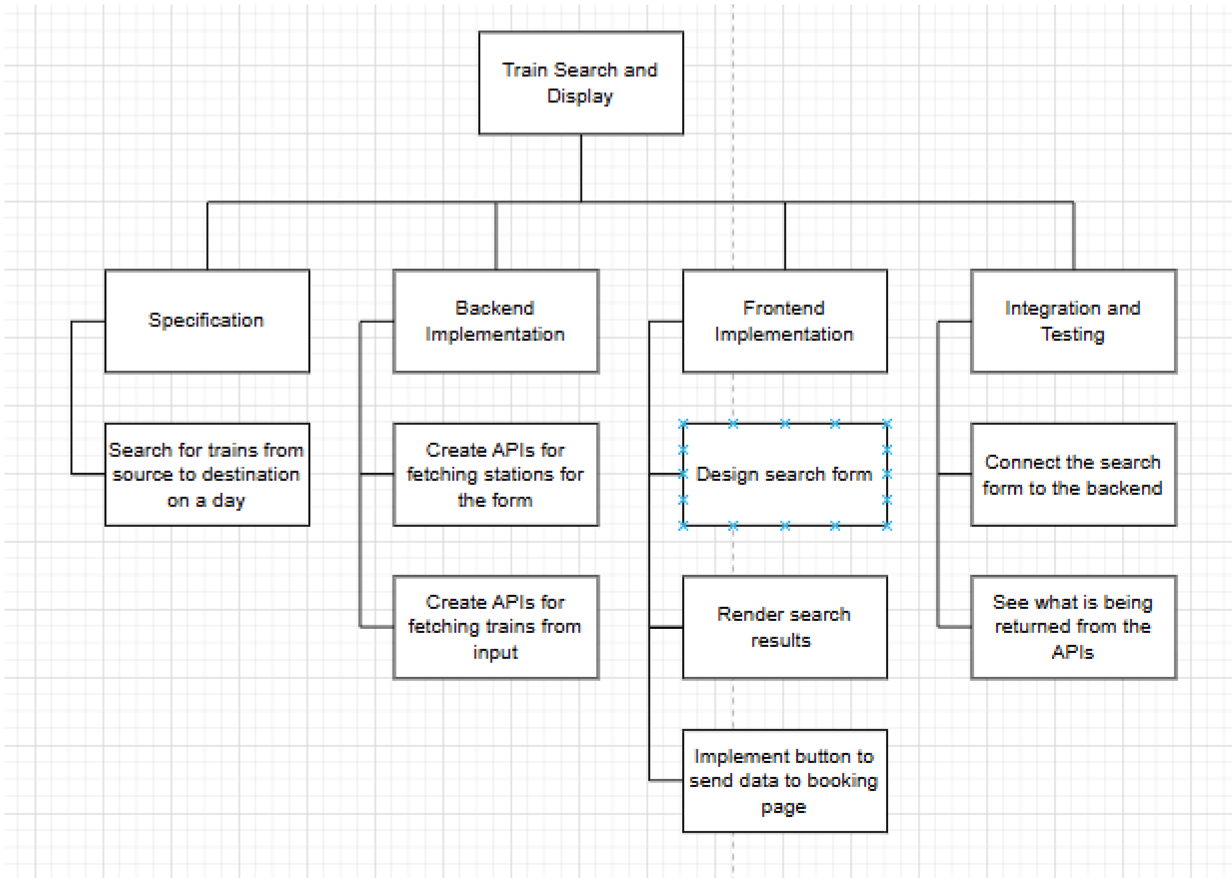


Figure 3: Train Search & Display Tasks

3.3 The tasks for Coach & Seat Selection are shown in Figure 4:

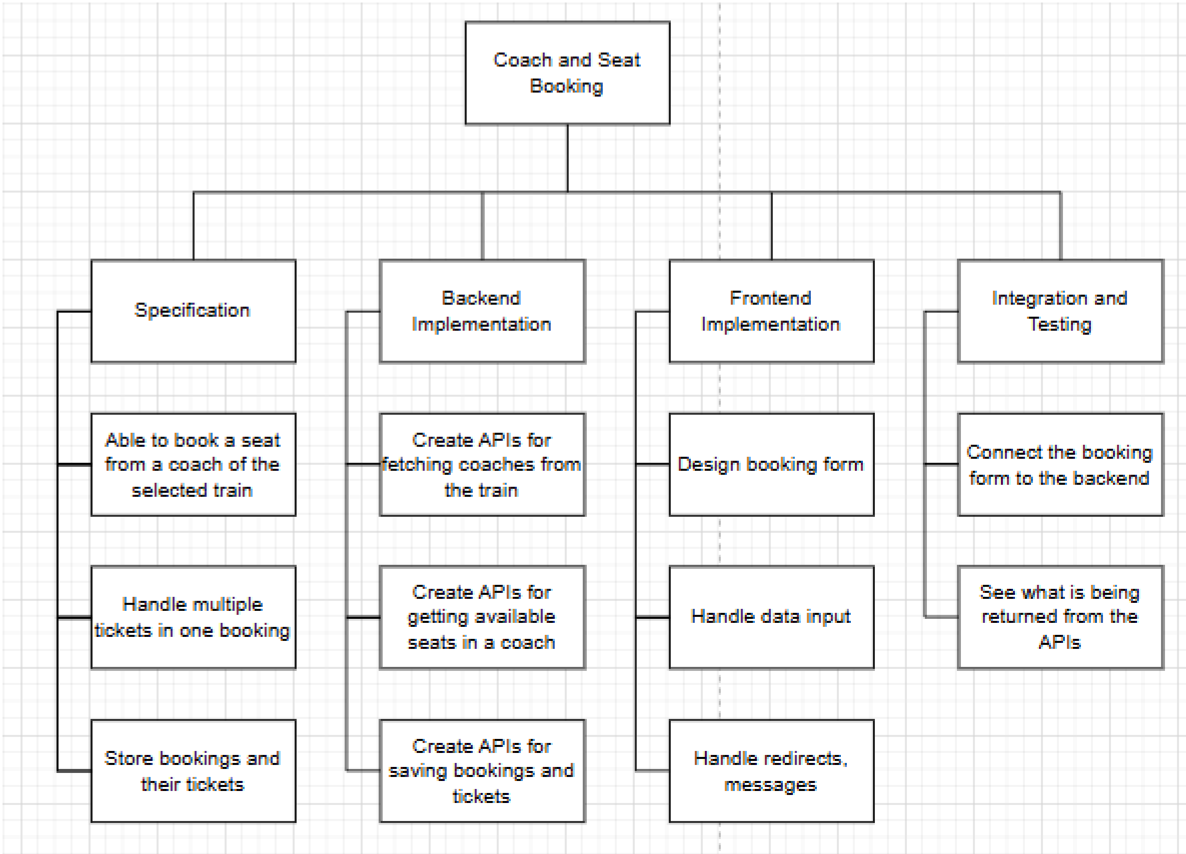


Figure 4: Coach & Seat Selection Tasks

3.4 The tasks for Booking Management are shown in Figure 5:

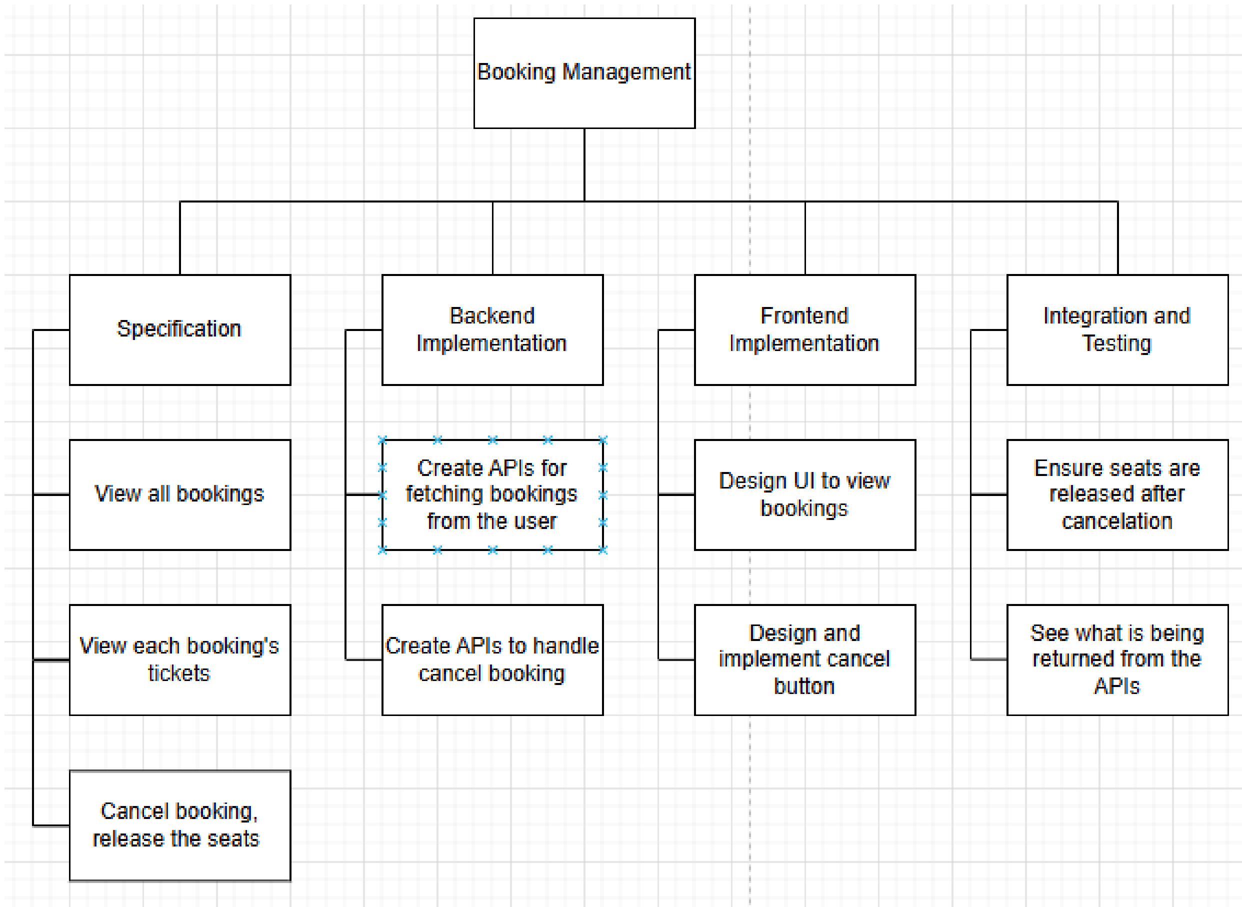


Figure 5: Booking Management Tasks

These tasks are expected to be implemented and validated during the development process, as we follow an incremental approach that emphasizes continuous integration and testing.

4. DEVELOPMENT PROCESS

For this project, our team has chosen to follow an **incremental and iterative development approach**. This approach fits our goal of building the system feature by feature, allowing us to adapt to changes in requirements and continuously improve the product.

Since we are still in the early stages of our software development journey, building the project one feature at a time helps us stay focused and learn effectively. After implementing each major feature, we integrate it into the system and test the functionality thoroughly. We also improve the user interface based on feedback and testing results. This allows us to track progress clearly and make adjustments as we discover better solutions or encounter new challenges.

5. DEVELOPMENT ENVIRONMENT

Our project is developed using modern web technologies and follows the **Model-View-Controller (MVC)** architecture to ensure maintainability and separation of concerns.

The following technologies are used in our system:

5.1 Backend Technologies:

- **Java 21** – Modern Java version with latest features and improvements
- **Spring Boot 3.5.7** – Serves as the backend framework to handle routing, business logic, and RESTful API endpoints (Controller layer)
- **Spring Data JPA** – Provides ORM functionality for database operations (Model layer)
- **PostgreSQL** – Primary relational database for production environment
- **Spring Security (BCrypt)** – Used for password encryption and security
- **Maven** – Build automation and dependency management tool

5.2 Frontend Technologies:

- **HTML5, CSS3, JavaScript** – Core web technologies for building the user interface (View layer)
- **Responsive Design** – Mobile-friendly UI that works across different screen sizes

5.3 Development Tools:

- **VS Code** – Primary code editor
- **Git & GitHub** – Version control and collaboration
- **Postman** – API testing and documentation

The system architecture follows the **MVC pattern** (see Figure 6):

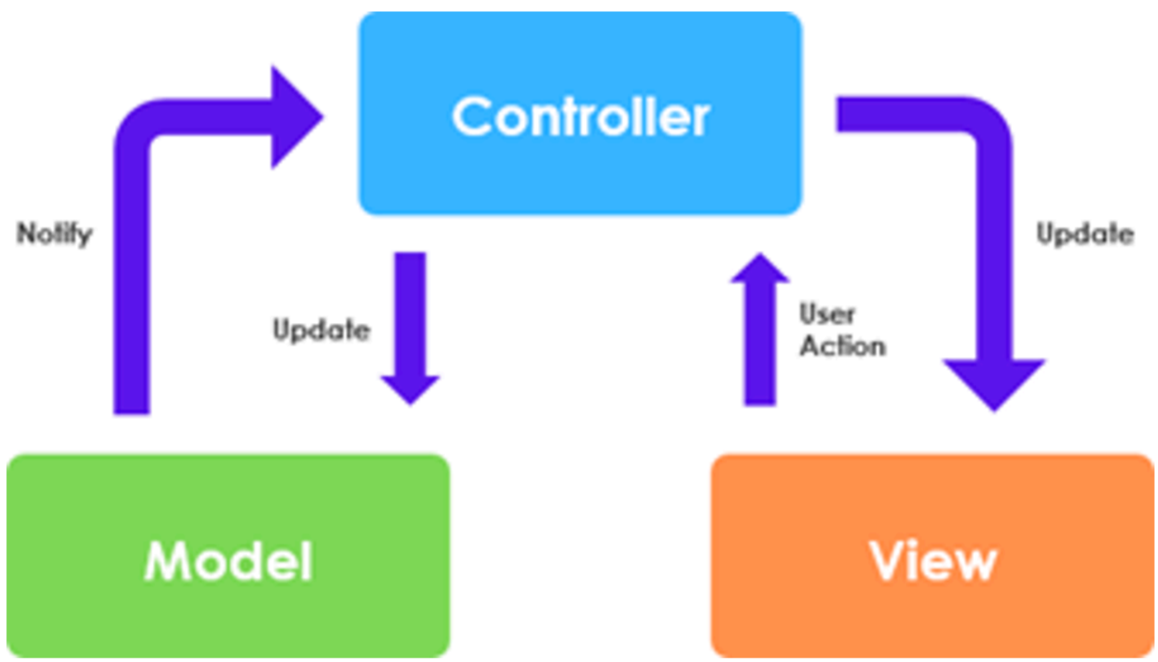


Figure 6: System Architecture

- The **Model** handles data and schema definitions (e.g., User, Train, Booking, Coach, Seat, Ticket, Station, Schedule)
- The **View** consists of HTML pages and JavaScript that display data and collect user input
- The **Controller** processes user actions, interacts with the service layer, and returns the appropriate response to the view
- The **Service** layer contains business logic and coordinates between controllers and repositories
- The **Repository** layer provides data access abstraction using Spring Data JPA

This environment ensures a modular and scalable structure, suitable for an iterative development approach.

We also utilize UML tools to assist in designing essential diagrams such as the Entity Relationship Diagram (ERD) and Use Case Diagram, which help us visualize the system's structure and user interactions effectively.

II. REQUIREMENT ANALYSIS AND DESIGN

This section outlines the process of analyzing requirements and designing the system. It serves as a foundation for the upcoming implementation phase of the project. Using this version of the requirements specification, we will develop each feature while ensuring all conditions, as well as functional and non-functional requirements provided by the users, are met. Throughout development, we will continuously refine and update this document to accurately reflect the current progress and direction of the project.

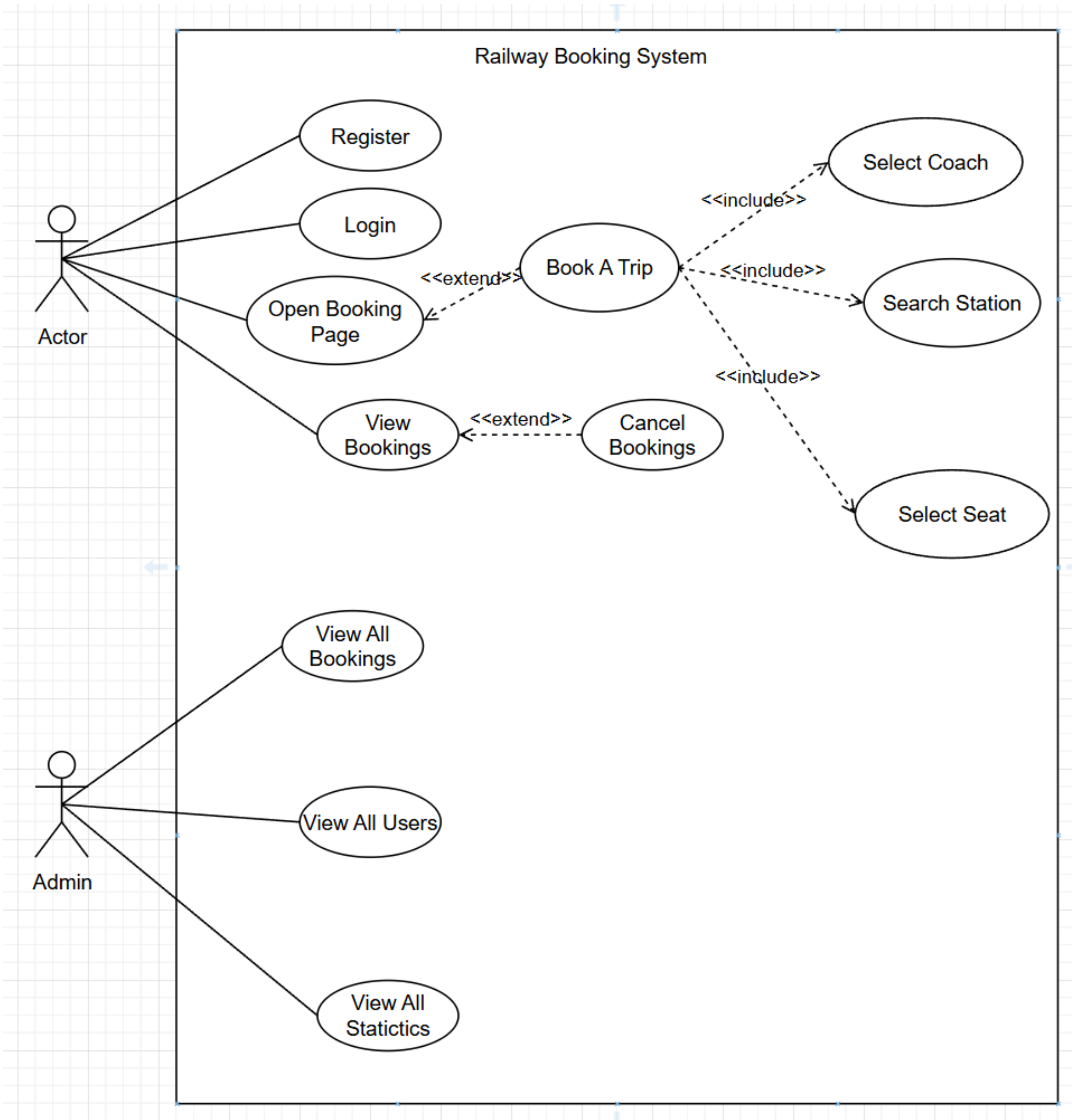
1. Requirement Analysis

1.1 USE CASE DIAGRAM

The use case diagram below illustrates the key actors and their interactions with the Railway Booking System. The system supports two primary actors:

- **User:** Regular users who can register, login, search trains, book trips, view/cancel bookings

- **Admin:** System administrator who manages trains, stations, schedules, users, and bookings



1.2 Use Case 1: Register Account

Name: Register a Railway System Account

Identifier: UC1

Inputs: Username, Email, Phone, Password

Outputs: Confirmation message or error message

Actor: Actor (User)

Actor: User	System
Open registration page	Display the registration form
Enter username, email, phone, password	
Click register button	Validate the user’s info Check for duplicate username/email/phone Encrypt password using BCryptCreate user account and redirect to login page

Precondition:

User has not registered before with the in-use email, username, or phone

Postcondition:

New account is created in the database

User Story: As a visitor, I want to register an account on the Railway Booking System so I can log in and book train tickets.

1.3 Use Case 2: Login to System

Name: Log in to Railway Booking System

Identifier: UC2

Inputs: Username, Password

Outputs: User is authenticated and redirected to home page (if succeed)User remains on login page with error message (if fail)

Actor: Actor (User)

Actor: User	System

Open login page	Display the login form
Enter username and password	
Click login button	Validate the user’s credentials Decrypt and verify password using BCrypt If succeed, create session and redirect to home pageIf fail, return error message

Precondition:

User has already registered an account

Postcondition:

User session is created (stored in localStorage/sessionStorage)

User Story: As a registered user, I want to log in so I can search for trains and book tickets.

1.4 Use Case 3: Open Booking Page

Name: Open Booking Page

Identifier: UC3

Inputs: Train selection from search results

Outputs: Booking page is displayed with train details and booking form

Actor: Actor (User)

Actor: User	System
Click “Book” button on a selected train	Redirect to booking pagePass train details (ID, name, source, destination, date, times) as query parameters Display booking form with train information

Precondition:

1. User has searched for trains

2. User has selected a train from search results

Postcondition:

Booking page is displayed with selected train details

User Story: As a user, I want to open the booking page for a selected train so I can proceed with ticket reservation.

Relationships: - Extended by: Book A Trip

1.5 Use Case 4: Book A Trip

Name: Book a Train Trip

Identifier: UC4

Inputs: Passenger information, Coach selection, Seat selection, Station information, Payment method

Outputs: Booking confirmation with booking reference number

Actor: Actor (User)

Actor: User	System
On booking page, select coach	Fetch available coaches Display coach options with availability
Select seat	Fetch available seats for selected coachDisplay seat map
Search/select station	Provide station search/autocomplete Validate station selection
Enter passenger details	

Click “Confirm & Pay”	Validate all input dataValidate coach and seat availability Calculate total fare Create booking record in database Generate ticketsDisplay success message with booking reference

Precondition:

1. User must be logged in

2. User must have opened booking page

3. Selected coach and seat must be available

Postcondition:

1. Booking record is created in database

2. Tickets are generated and linked to booking

3. Seats are marked as booked for the travel date

User Story: As a logged-in user, I want to book a trip by selecting coach, seat, and providing passenger information so I can confirm my train reservation.

Relationships: - Extends: Open Booking Page - Includes: Select Coach, Select Seat, Search Station

1.6 Use Case 5: Select Coach

Name: Select Train Coach

Identifier: UC5

Inputs: Train ID, Travel date

Outputs: List of available coaches with seat availability

Actor: Actor (User) — (included in Book A Trip)

Actor: User	System

View available coaches	Fetch all coaches for the selected train Calculate available seats per coach based on travel date Display coaches with availability information
Select a coach	Update UI to show selected coach Enable seat selection

Precondition:

1. User must be in booking flow

2. Train must be selected

Postcondition:

Selected coach information is stored for booking

User Story: As a logged-in user, I want to select a coach so that I can choose my preferred seat category.

1.7 Use Case 6: Select Seat

Name: Select Seat Number

Identifier: UC6

Inputs: Coach ID, Travel date

Outputs: List of seats with availability status (available/booked)

Actor: Actor (User) — (included in Book A Trip)

Actor: User	System
View available seats in selected coach	Fetch all seats for the selected coach Check booking status for each seat on travel date Display seat map with availability

Select an available seat	Validate seat availability Update UI to mark seat as selected Store seat selection

Precondition:

1. User must be in booking flow

2. Coach must be selected

Postcondition:

Selected seat information is stored for booking

User Story: As a logged-in user, I want to select a specific seat number so that I can reserve my preferred seating position.

1.8 Use Case 7: Search Station

Name: Search Railway Station

Identifier: UC7

Inputs: Station name (partial or full)

Outputs: List of matching stations

Actor: Actor (User) — (included in Book A Trip)

Actor: User	System
Enter station name in search/autocomplete field	Query database for stations matching input Display filtered list of stations
Select a station from results	Store selected station for booking

Precondition:

User is in train search or booking flow

Postcondition:

Station selection is stored

User Story: As a user, I want to search for stations so that I can select the correct source and destination for my journey.

1.9 Use Case 8: View Bookings

Name: View My Bookings

Identifier: UC8

Inputs: User ID (from session)

Outputs: List of user's bookings with details

Actor: Actor (User)

Actor: User	System
Navigate to "My Bookings" page	Fetch all bookings for the logged-in user Retrieve associated ticket and train information Display bookings in organized list with booking details

Precondition:

User must be logged in

Postcondition:

None

User Story: As a logged-in user, I want to view my booking history so that I can track my train reservations.

Relationships: - Extended by: Cancel Bookings

1.10 Use Case 9: Cancel Bookings

Name: Cancel Train Booking

Identifier: UC9

Inputs: Booking ID

Outputs: Cancellation confirmation message

Actor: Actor (User)

Actor: User	System
From “My Bookings” page, select a booking	Display booking details
Click “Cancel Booking” button	Validate booking cancellation eligibility Mark booking as cancelled Free up booked seats Process refund (if applicable) Display cancellation confirmation

Precondition:

1. User must be logged in

2. Booking must exist and belong to user

3. Booking must be cancellable (based on travel date)

Postcondition:

1. Booking status is updated to cancelled

2. Seats are released and become available again

User Story: As a logged-in user, I want to cancel my booking so that I can free up my reservation if my plans change.

Relationships: - Extends: View Bookings

1.11 Use Case 10: View All Statistics (Admin)

Name: View All Railway Statistics

Identifier: UC10

Inputs: None

Outputs: Complete list of all railway statistics in the system

Actor: Admin

Actor: Admin	System
Navigate to dashboard page	Fetch all statistics from database Display complete station list with total bookings, total users, total trains, cancelled bookings

Precondition:

Admin must be logged in

Postcondition:

None

User Story: As an admin, I want to view all statistics so that I can manage the station database.

1.12 Use Case 11: View All Users (Admin)

Name: View All System Users

Identifier: UC19

Inputs: None

Outputs: Complete list of all registered users

Actor: Admin

Actor: Admin	System
Navigate to users management page	Fetch all users from database Display user list with ID, username, email, phone, role

Precondition:

Admin must be logged in

Postcondition:

None

User Story: As an admin, I want to view all users so that I can manage user accounts and monitor system usage.

1.13 Use Case 12: View All Bookings (Admin)

Name: View All Bookings

Identifier: UC20

Inputs: None (or optional filters: date range, train, user)

Outputs: Complete list of all bookings in the system

Actor: Admin

Actor: Admin	System
Navigate to bookings management page	Fetch all bookings from database Retrieve associated ticket, train, and user information Display bookings with details (booking ID, user, train, date, status)

Precondition:

Admin must be logged in

Postcondition:

None

User Story: As an admin, I want to view all bookings so that I can monitor reservations and system usage.

2. Design

Entity Relationship Diagram

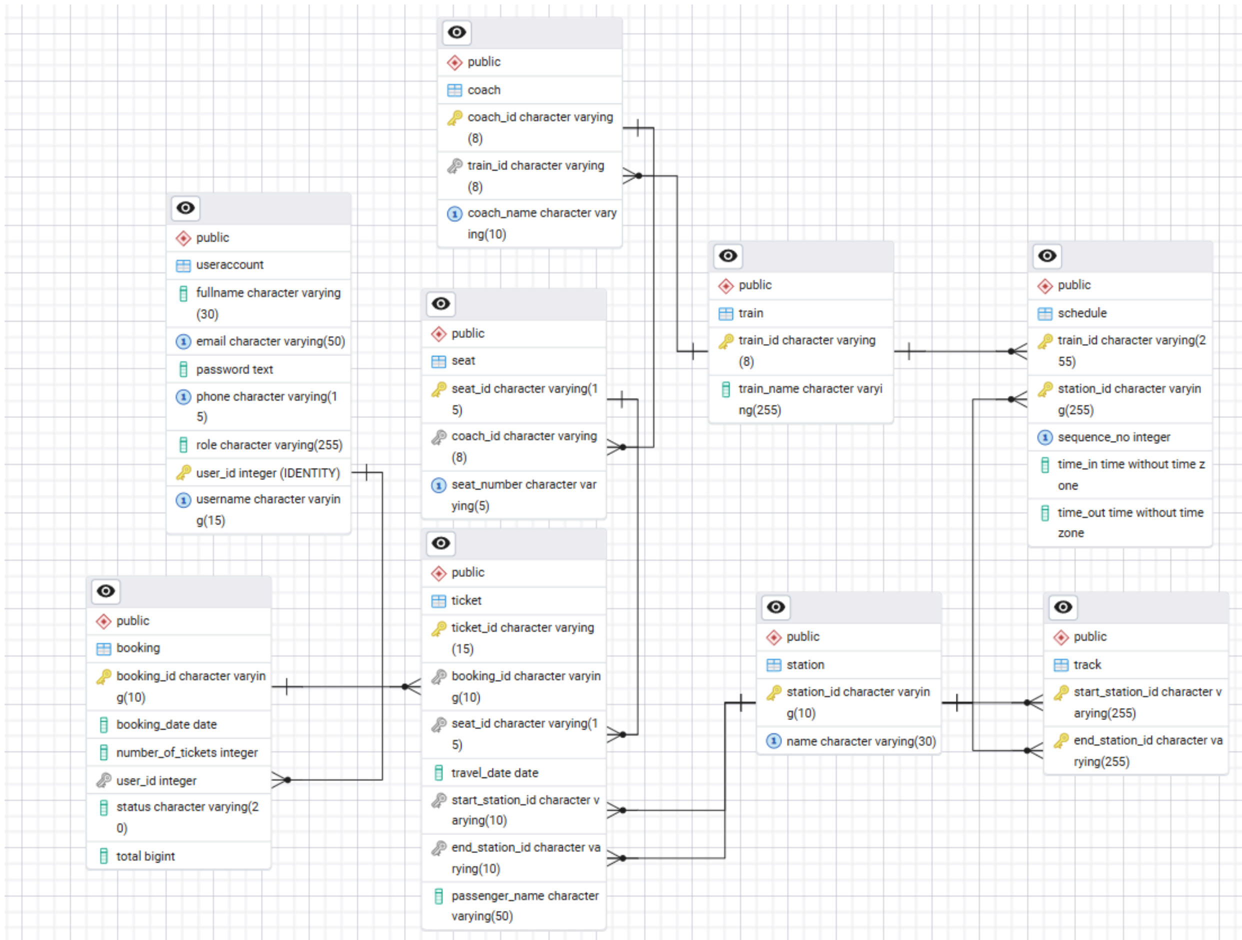


Figure 7: Entity Relationship Diagram

The system uses the following main entities in **Figure 7**:

- **User**: userId (PK), username, email, phone, password
- **Train**: trainId (PK), trainName, trainNumber
- **Station**: stationId (PK), stationName, location
- **Schedule**: scheduleId (PK), trainId (FK), stationId (FK), arrivalTime, departureTime, distance
- **Coach**: coachId (PK), trainId (FK), coachName, coachType
- **Seat**: seatId (PK), coachId (FK), seatNumber, seatType
- **Booking**: bookingId (PK), userId (FK), bookingDate, numberOfTickets, total
- **Ticket**: ticketId (PK), bookingId (FK), seatId (FK), passengerName, travelDate, startStationId (FK), endStationId (FK)

III. IMPLEMENTATION

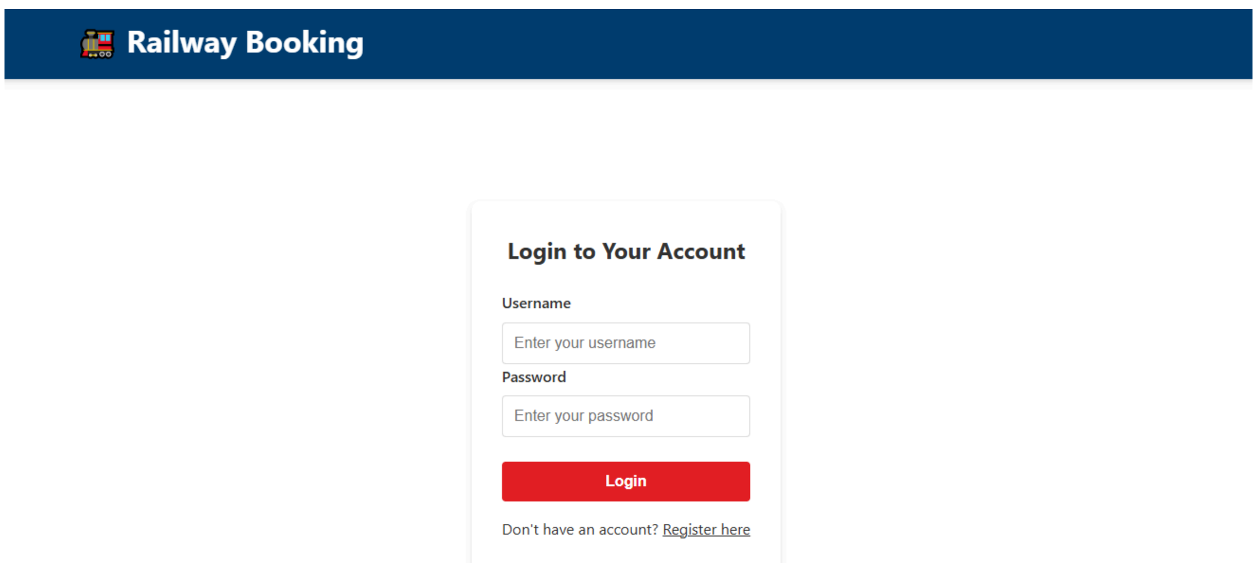
1. USER'S ACCOUNT MANAGEMENT FUNCTIONS

1.1. Register a New Account

Step 1: Access the Railway Booking System homepage via the URL:

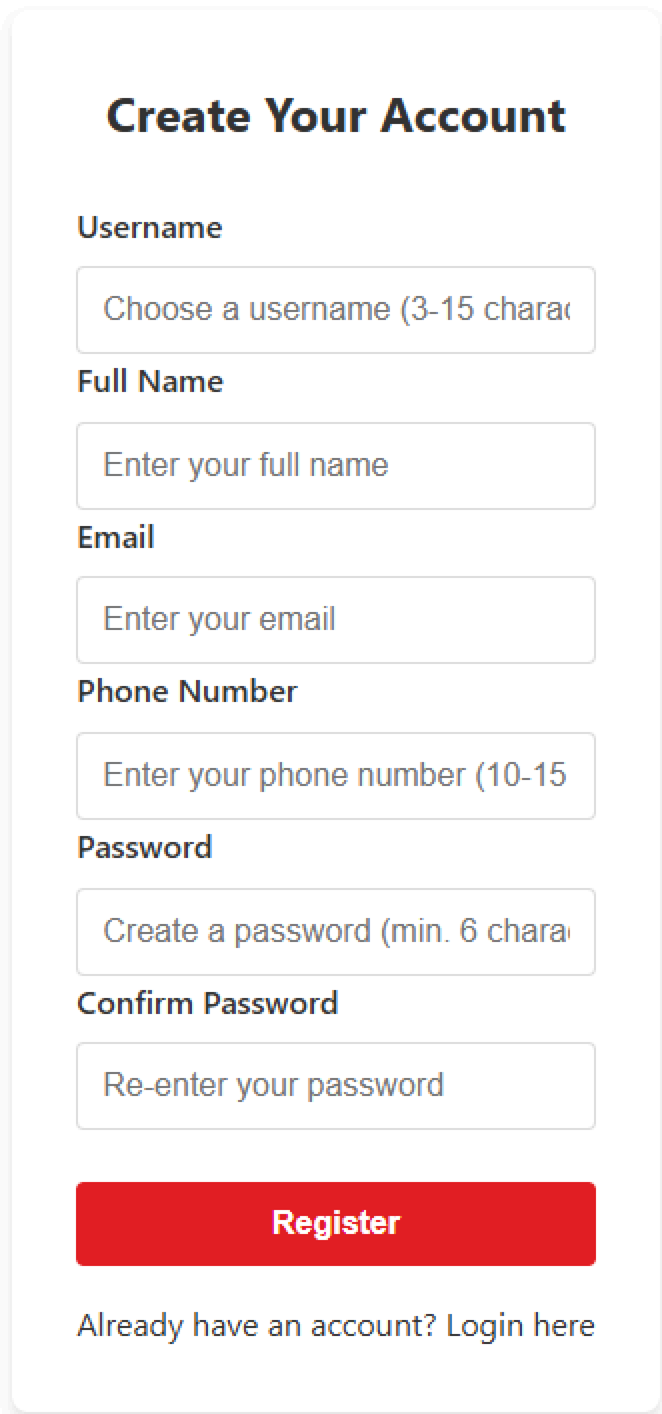
http://localhost:8081/

Click on the **Register** button or link from the homepage to navigate to the registration page.



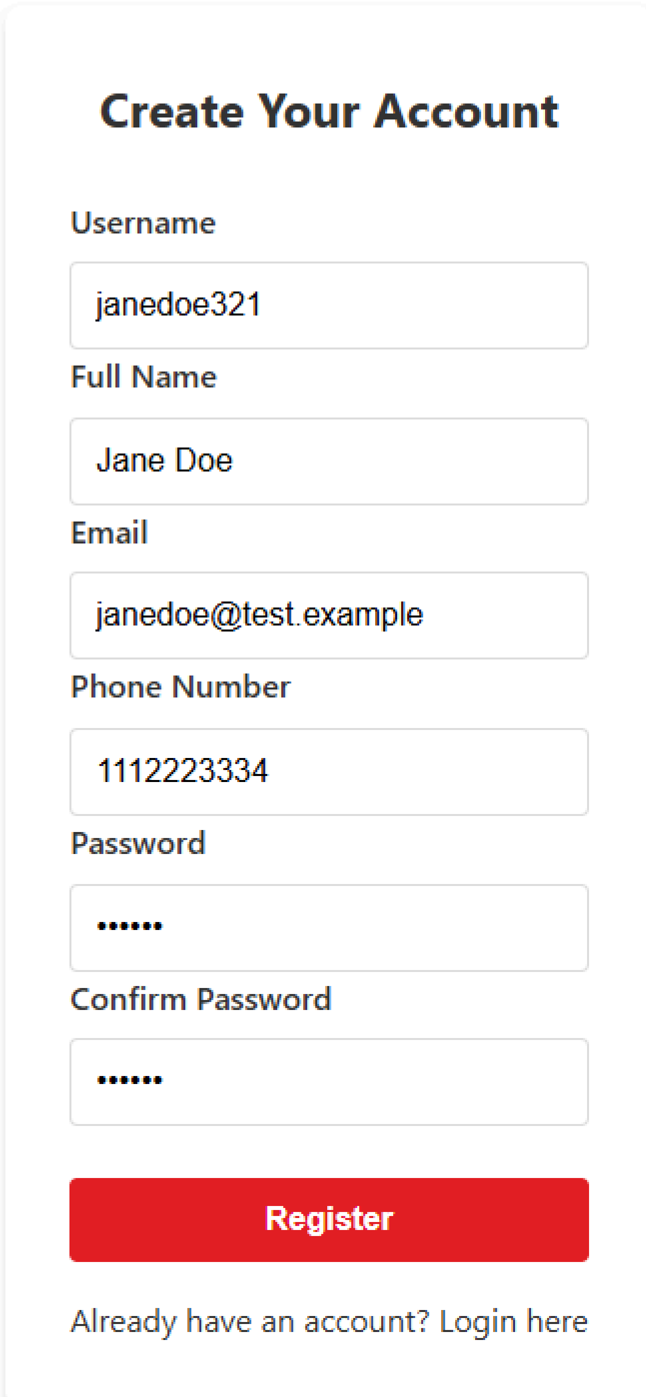
Step 2: The registration page will appear with a form requesting the following information:

- **Username** (required, unique)
- **Full Name** (required)
- **Email** (required, unique, valid format)
- **Phone Number** (required, unique)
- **Password** (required, minimum 6 characters)
- **Confirm Password** (required, must match password)



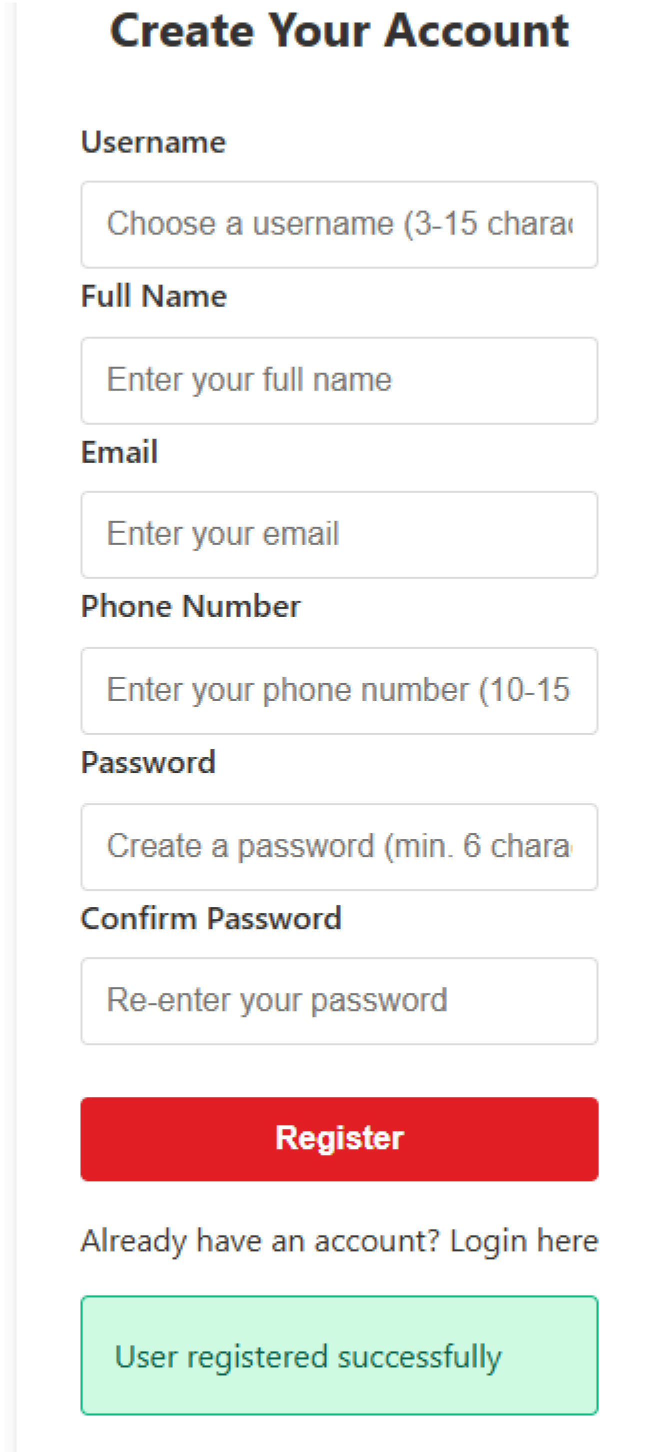
Step 3: Fill in all the required fields with valid information.

Note: - Fields marked with an asterisk (*) are mandatory - Username must be unique and cannot contain special characters - Email must be in valid format (example@domain.com) - Phone number must be in valid format (10-15 digits) - Password must be at least 6 characters long and will be encrypted using BCrypt

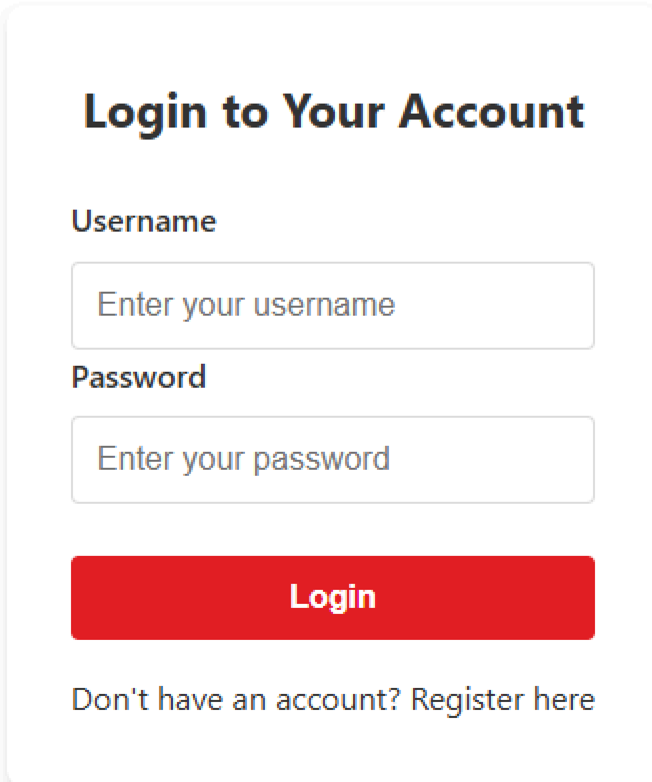


Step 4: Click the **"Register"** button to submit the registration form.

If all validations pass, a success message will appear confirming that the account has been created successfully.



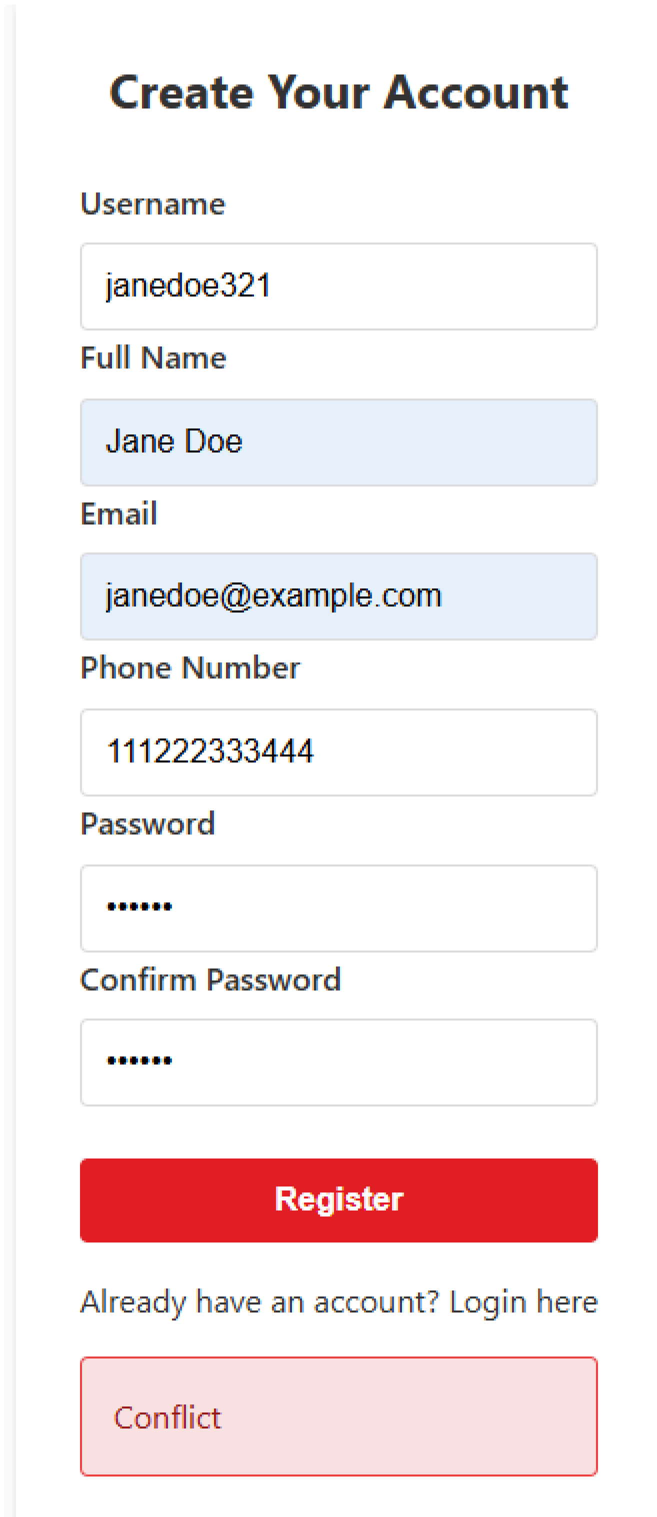
Step 5: After successful registration, you will be automatically redirected to the login page to sign in with your newly created account.



Error Handling:

If the registration fails due to validation errors, appropriate error messages will be displayed:

- **Duplicate Username:** “Username already exists. Please choose a different username.”
- **Duplicate Email:** “Email already exists. Please use a different email address.”
- **Duplicate Phone:** “Phone number already exists. Please use a different phone number.”
- **Invalid Format:** Field-specific validation messages (e.g., “Invalid email format”)

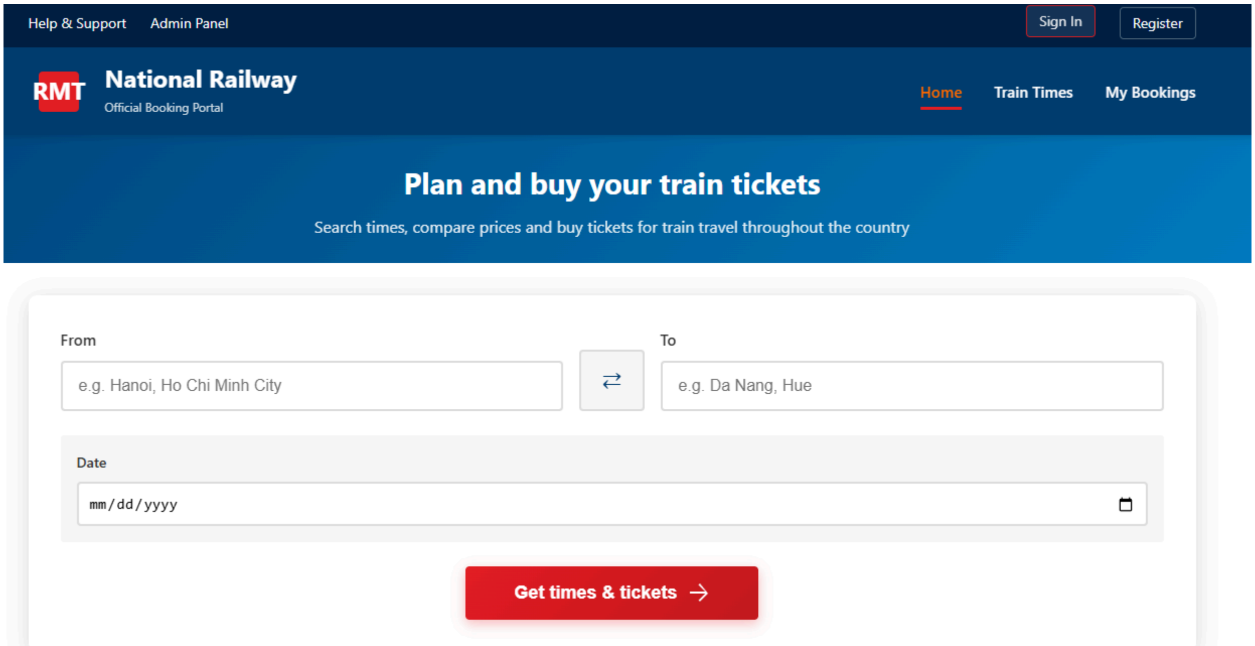


1.2. Login to the System

Step 1: From the homepage, click on the **“Login”** button.

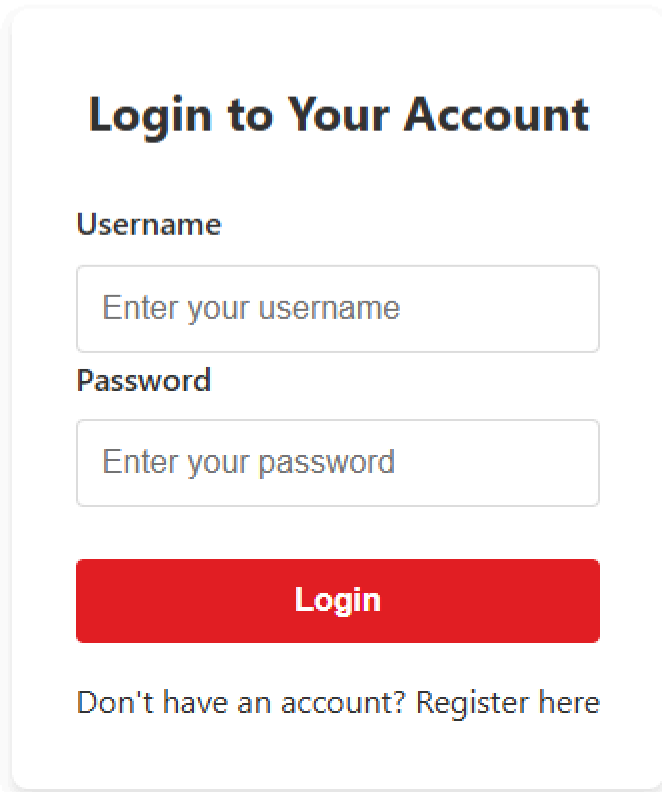
Alternatively, access the login page directly via:

http://localhost:8080/login.html

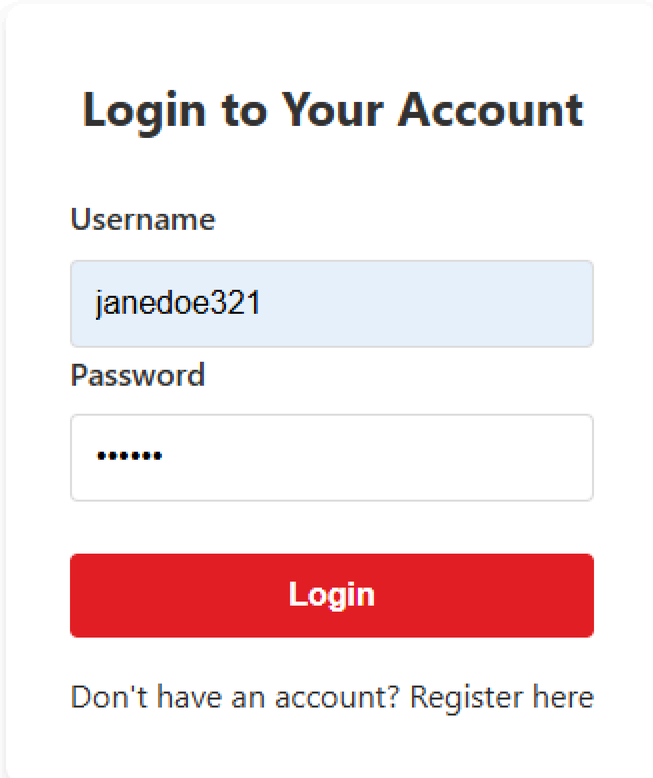


Step 2: The login page will appear with a form requesting:

- **Username**
- **Password**



Step 3: Enter your registered username and password in the respective fields.



Step 4: Click the **"Login"** button to submit your credentials.

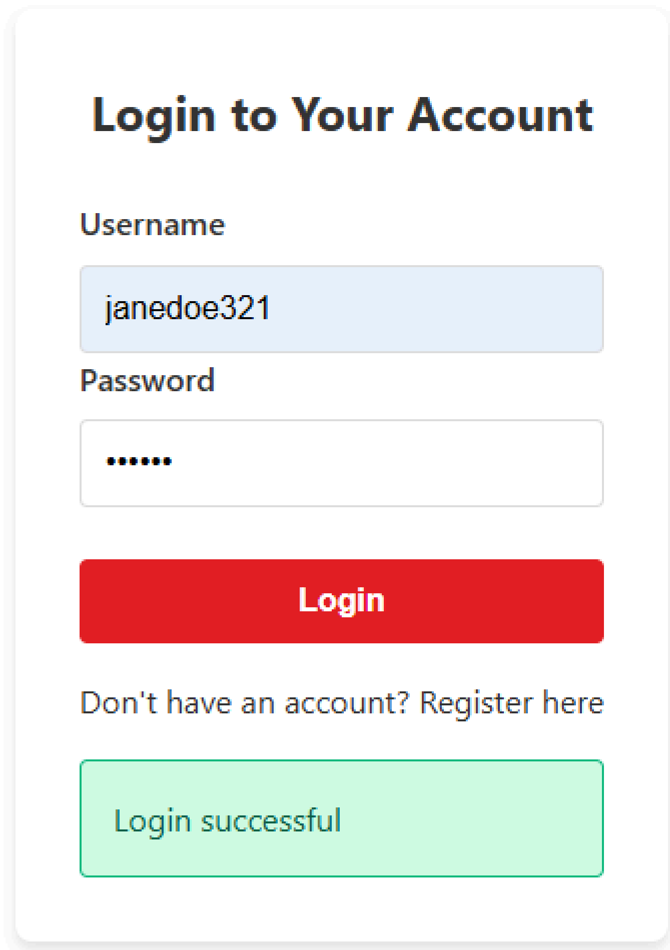
If the credentials are valid, the system will:

1. Verify the password using BCrypt decryption

2. Create a user session (stored in browser localStorage)

3. Display a success message

4. Redirect you to the homepage

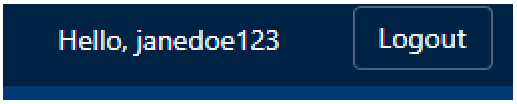


Step 5: After successful login, you will see your username displayed in the navigation bar, and additional options will become available:

- **Search Trains**
- **My Bookings** (to view your booking history)
- **Logout**

1.3. Logout from the System

Step 1: After logging in, locate the **Logout** button in the navigation bar (typically at the top-right corner).



Step 2: Click the **Logout** button.

The system will:

1. Clear the user session from localStorage

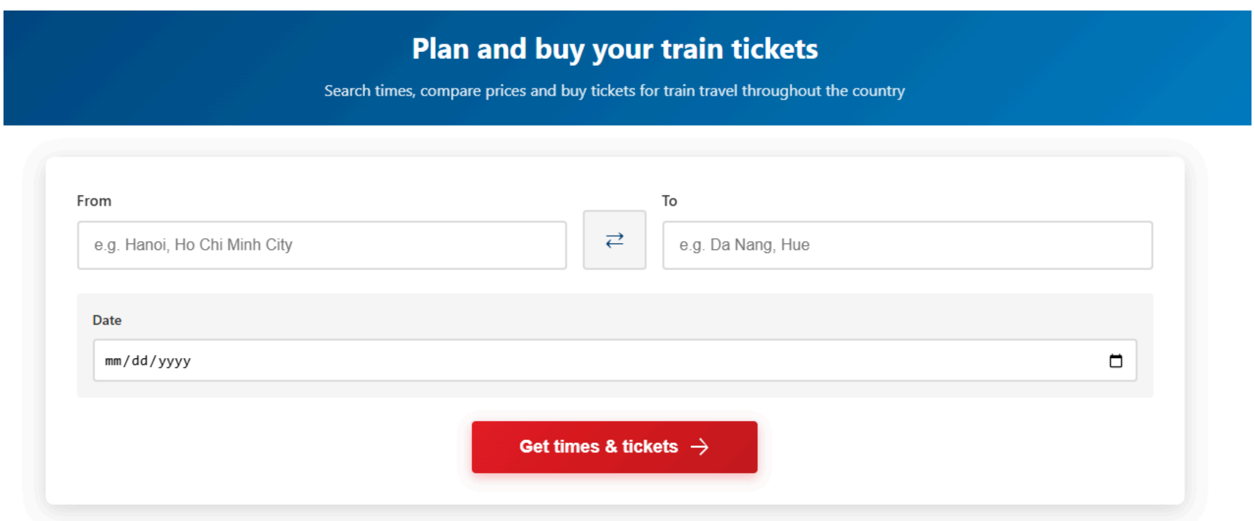
2. Redirect you to the homepage

2. TRAIN SEARCH & BOOKING FUNCTIONS

2.1. View All Stations

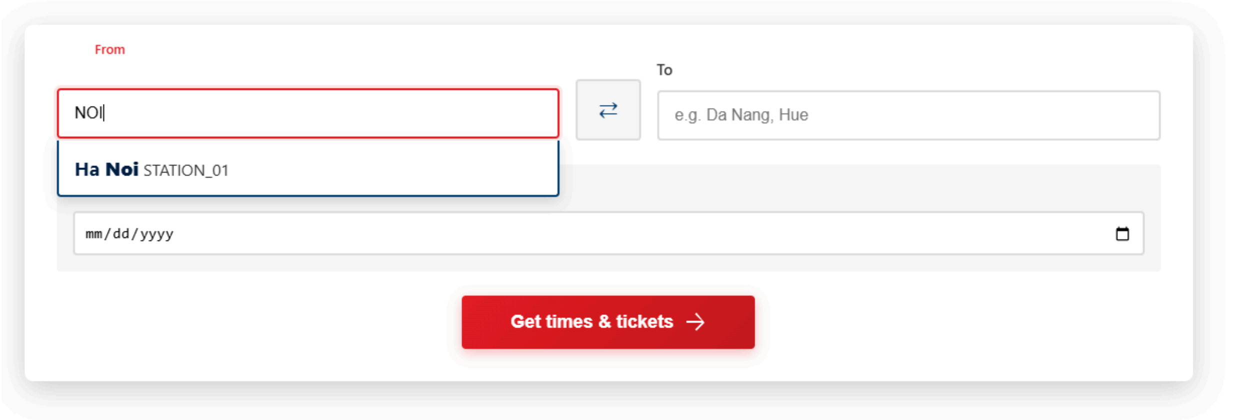
Step 1: From the homepage, navigate to the train search section.

The system provides a list of all available railway stations for selection. Stations can be accessed via: - Autocomplete dropdown when typing in search fields - Direct API endpoint: GET /api/stations



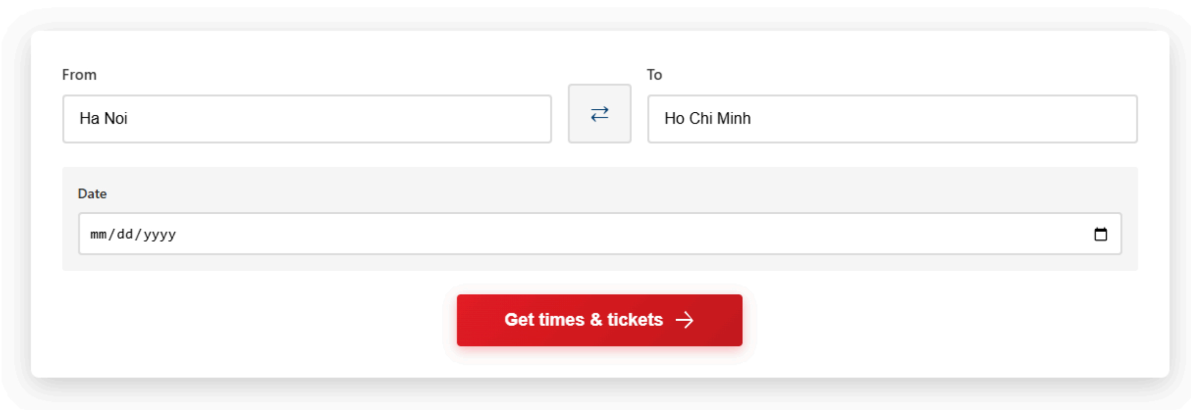
Step 2: Click on the **“Source Station”** or **“Destination Station”** input field.

As you start typing, an autocomplete dropdown will appear showing matching stations.



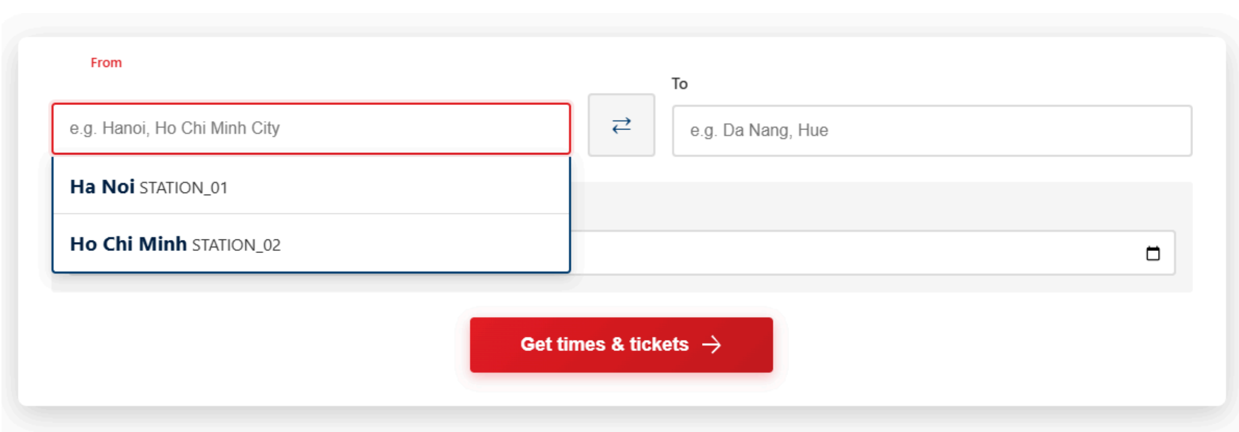
Step 3: Select a station from the dropdown list by clicking on it.

The selected station will populate the input field.



Station List Information:

All stations are fetched from the database and displayed in alphabetical order. Each station entry includes: - **Station ID** - **Station Name** (e.g., Hanoi, Ho Chi Minh City, Da Nang)

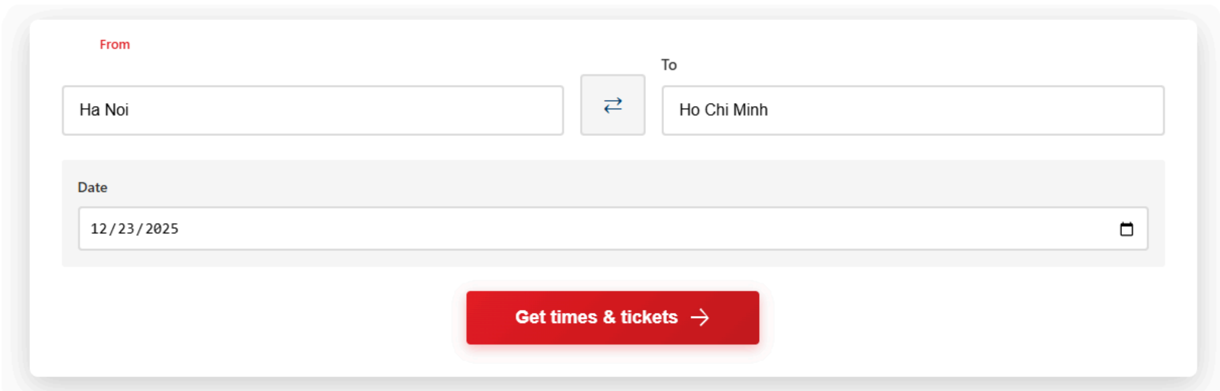


2.2. Search Trains by Route

Step 1: From the homepage, locate the **“Train Search”** section.

Step 2: Fill in the search criteria:

- **Source Station** (required) - Select departure station from autocomplete
- **Destination Station** (required) - Select arrival station from autocomplete
- **Travel Date** (optional) - Select date using date picker



Step 3: Click the **“Search Trains”** button to submit the search request.

The system will:

1. Validate that source and destination stations are different

2. Query the database for trains traveling between the selected stations

3. Fetch schedule information (departure/arrival times)

4. Calculate journey duration

5. Display results

Step 4: The search results page will display a list of available trains with the following information for each train:

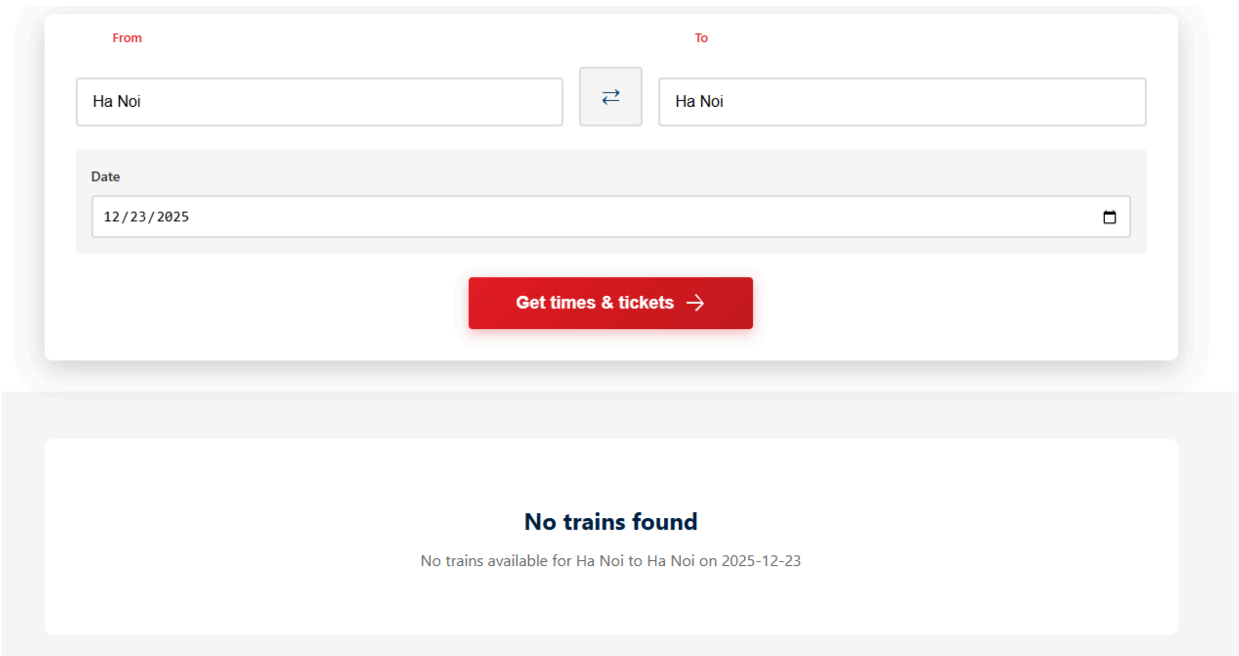
- **Train Number** (e.g., SE1, SE3)
- **Train Name** (e.g., Reunification Express)
- **Departure Time** (time train leaves source station)
- **Arrival Time** (time train arrives at destination station)

- **Duration** (calculated journey time)
- **“Book Ticket”** button

Error Handling:

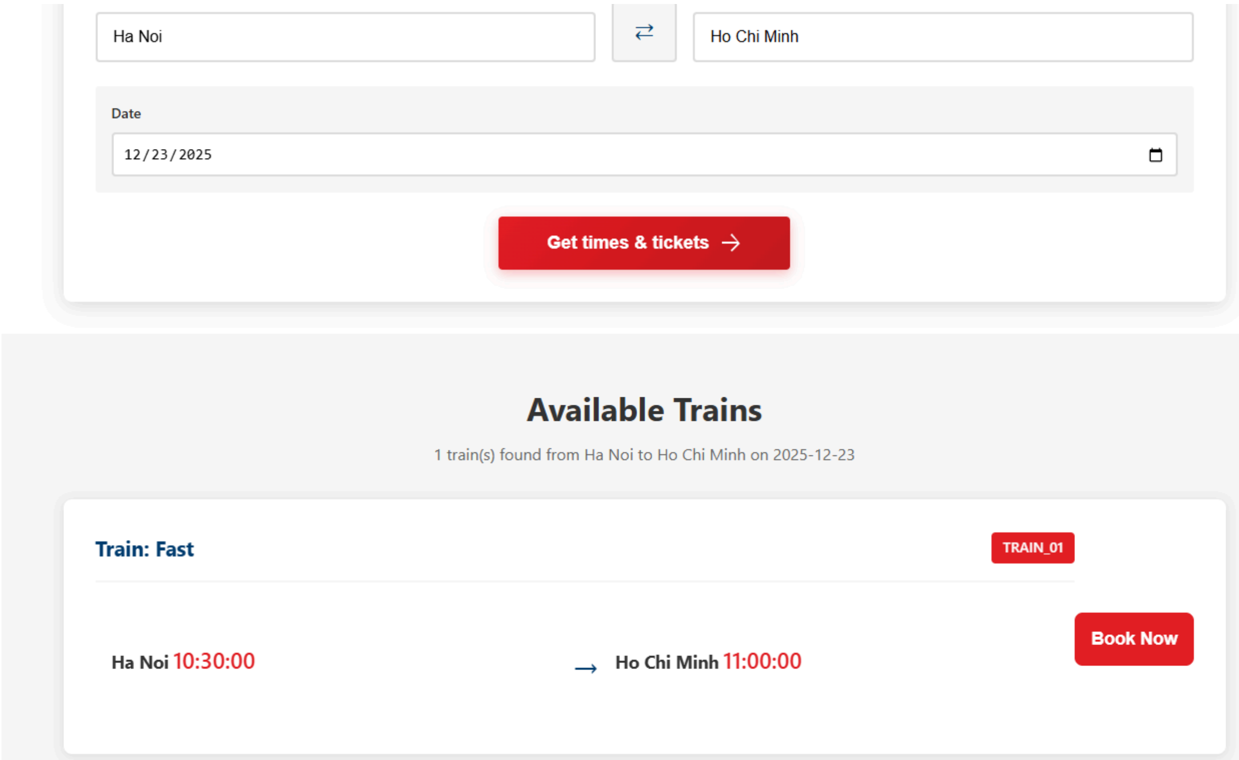
If no trains are found, a message will be displayed:

“No trains found. Please try again with different stations.”



2.3. View Train Details

Step 1: From the search results page, click on a train card to view details or proceed to booking.



Step 2: A detailed train information modal or page will appear showing:

- **Train ID** (e.g., TR001)
- **Train Number** (e.g., SE1)
- **Train Name** (e.g., Reunification Express)
- **Source Station**
- **Destination Station**
- **Departure Time**
- **Arrival Time**
- **Duration**

2.4. Open Booking Page

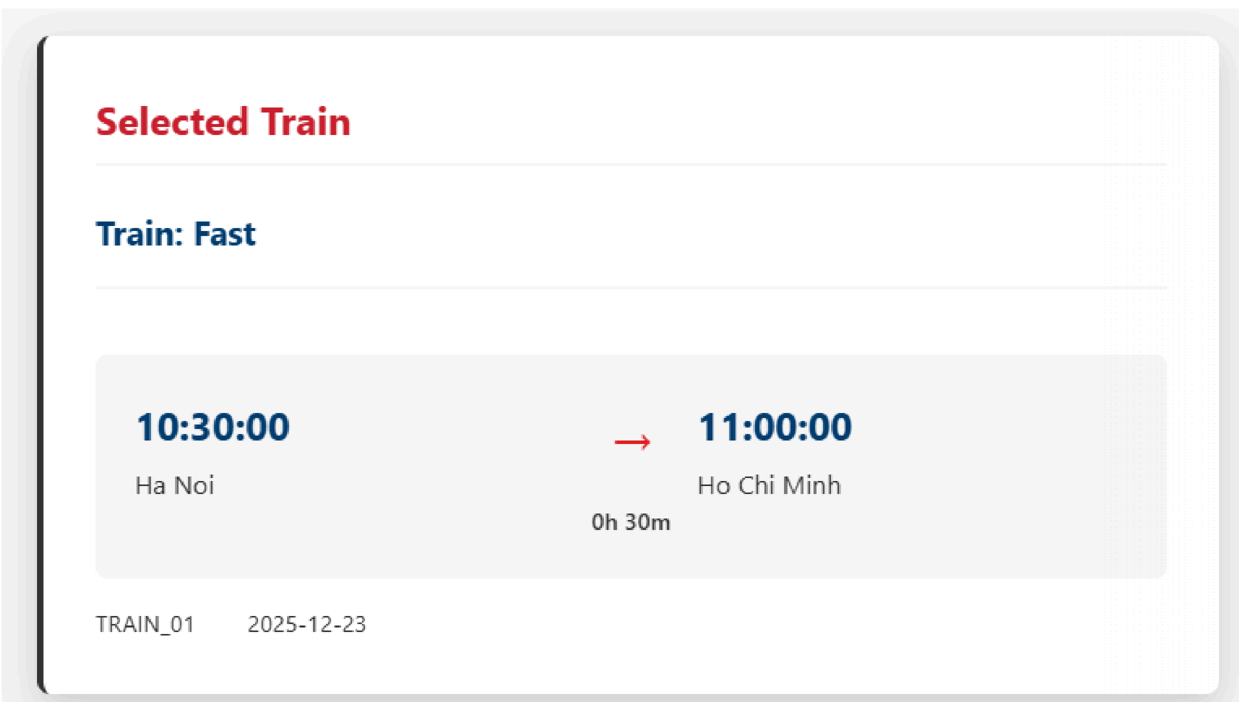
Step 1: After selecting a train from the search results, the system navigates to the booking page automatically.

Step 2: The system will redirect you to the booking page with the following train details pre-filled:

- **Train ID**
- **Train Name**
- **Source Station**
- **Destination Station**
- **Travel Date**
- **Departure Time**
- **Arrival Time**

The URL will include query parameters:

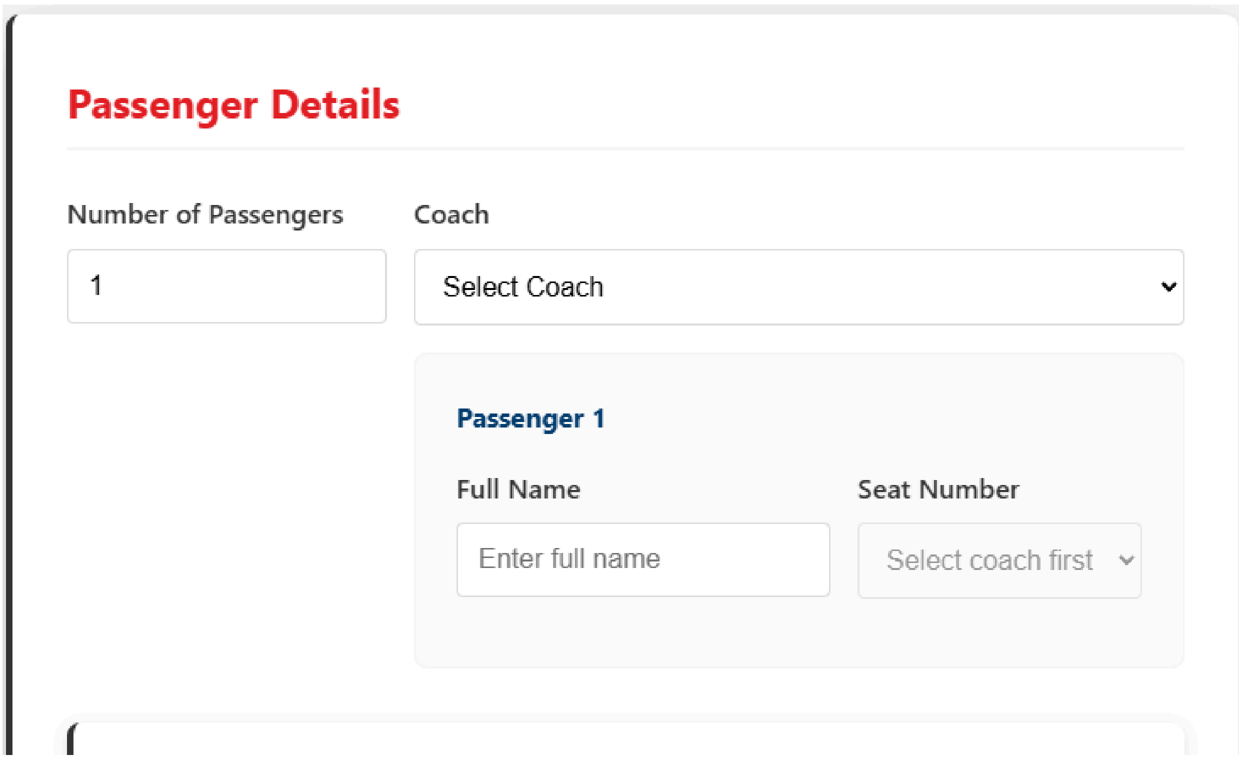
http://localhost:8080/booking.html?trainId=TR001&trainName=SE1&source=Hanoi&destination=HCMC&date=2025-01-15&departureTime=08:00&arrivalTime=20:00



Step 3: The booking page displays:

- **Train Journey Summary** (read-only)
- **Coach Selection** section

- **Seat Selection** section (enabled after coach selection)
- **Passenger Information** form
- **Confirm** button



2.5. Select Coach

Step 1: On the booking page, locate the **Select Coach** section.

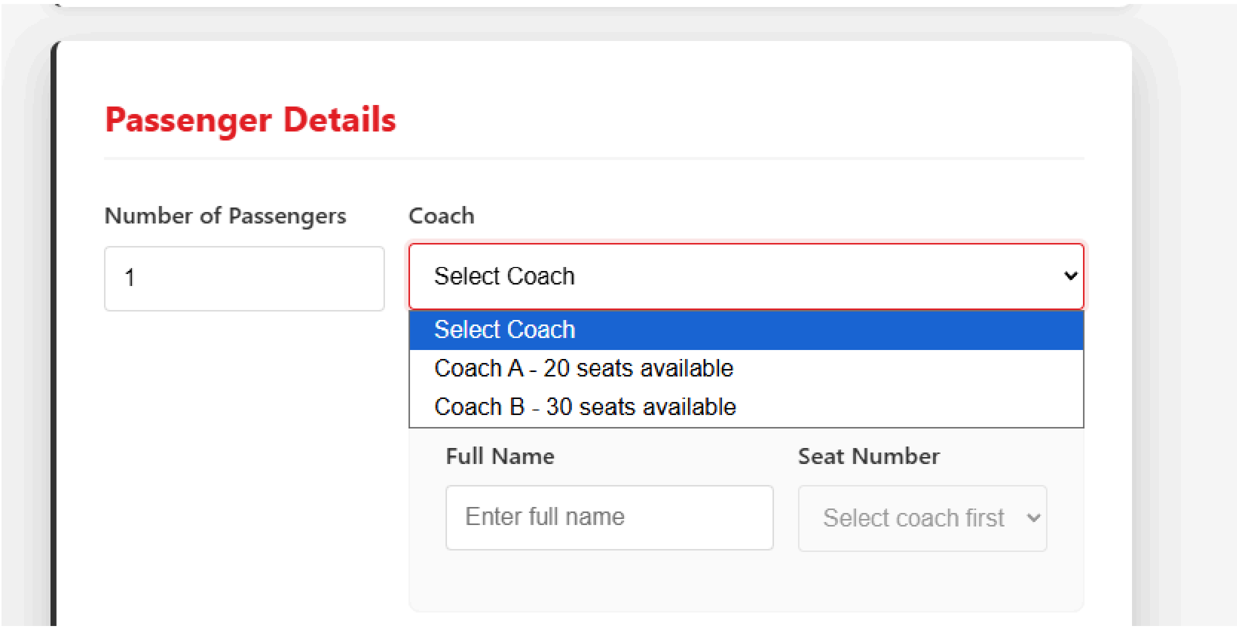
Step 2: Click the **View Coaches** button.

The system will make an API call:

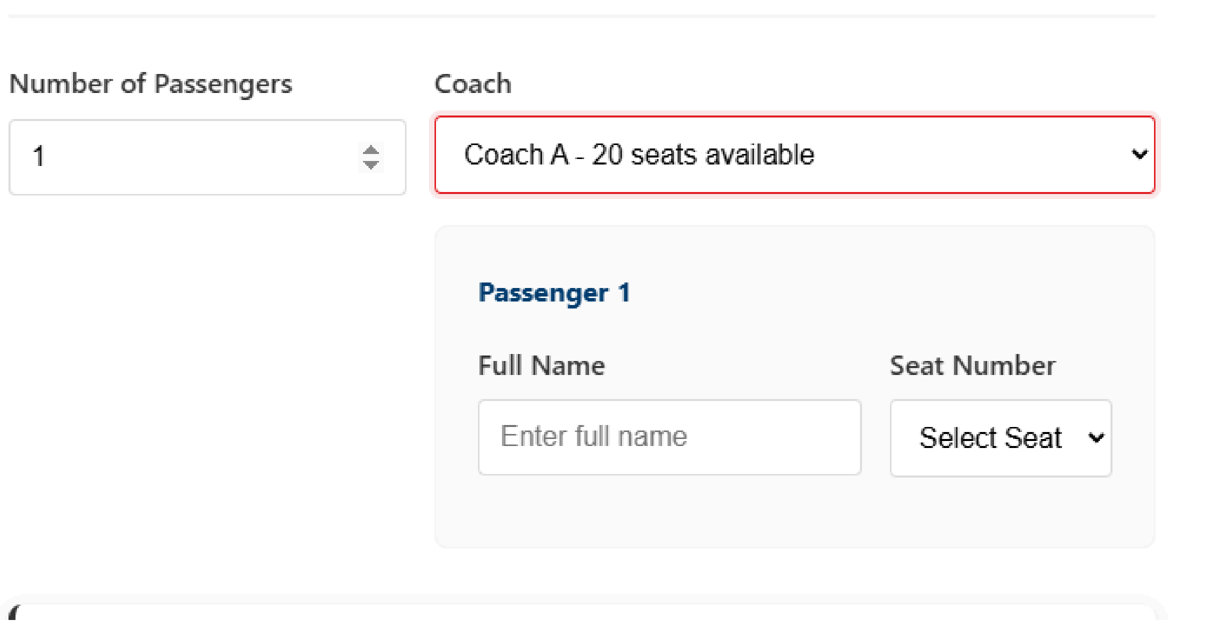
GET /api/coaches/{trainId}?date=YYYY-MM-DD

Step 3: A list of available coaches will be displayed, showing for each coach:

- **Coach ID**
- **Coach Name**
- **Available Seats**



Step 5: After selecting a coach, the **Select Seat** section will become enabled.



Coach Availability Calculation:

The system calculates available seats using the following logic:

1. Count total seats in the selected coach

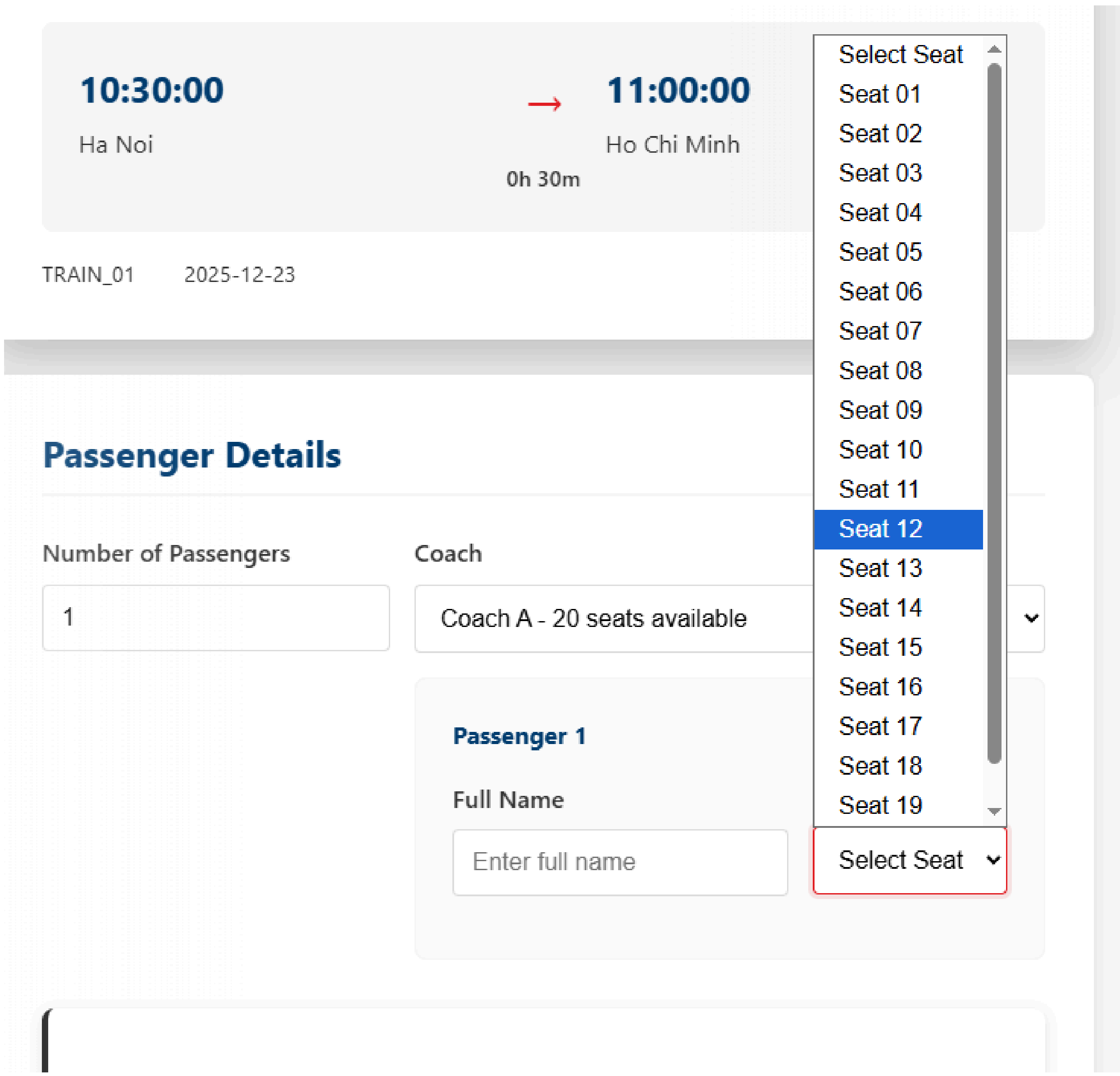
2. Query booked tickets for the coach on the selected travel date

3. Subtract booked seats from total seats

4. Display remaining available seats

2.6. Select Seat

Step 1: After selecting a coach, locate the **Select Seat** section.



Seat Availability Check:

The system checks seat availability in real-time:

1. Fetch all seats for the selected coach

2. Query the ticket table for bookings on the selected travel date

3. Mark seats as “Booked” if a ticket exists for that seat on that date

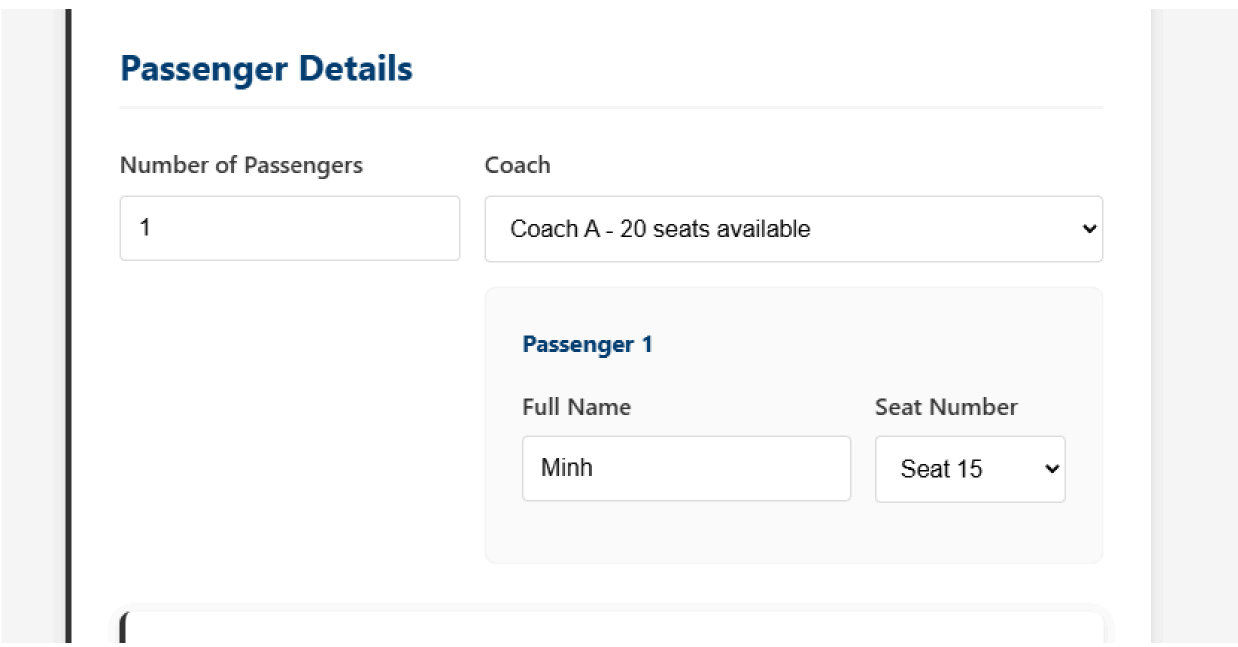
4. Display remaining seats as “Available”

2.7. Enter Passenger Information

Step 1: After selecting coach, the **“Passenger Details”** section is automatically visible on the booking form.

Step 2: Fill in the required passenger information:

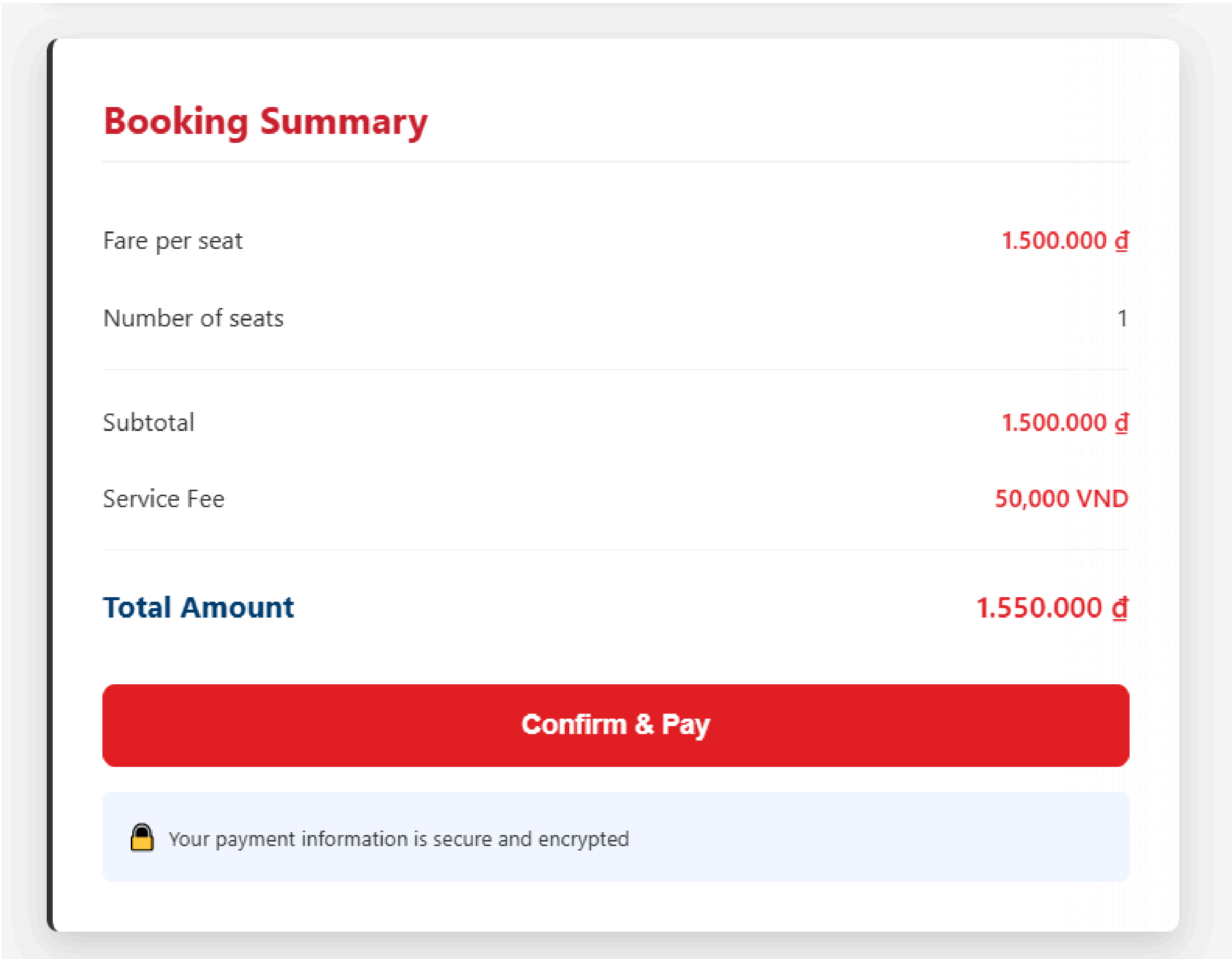
- **Passenger Name** (required)
- **Passenger Email** (optional, for confirmation)
- **Passenger Phone** (optional, for contact)



Step 3: Review the booking summary displaying:

- **Train Details** (number, name, route, time)
- **Coach and Seat** selection
- **Passenger Information**

- **Total Price** (calculated based on route and seat type)



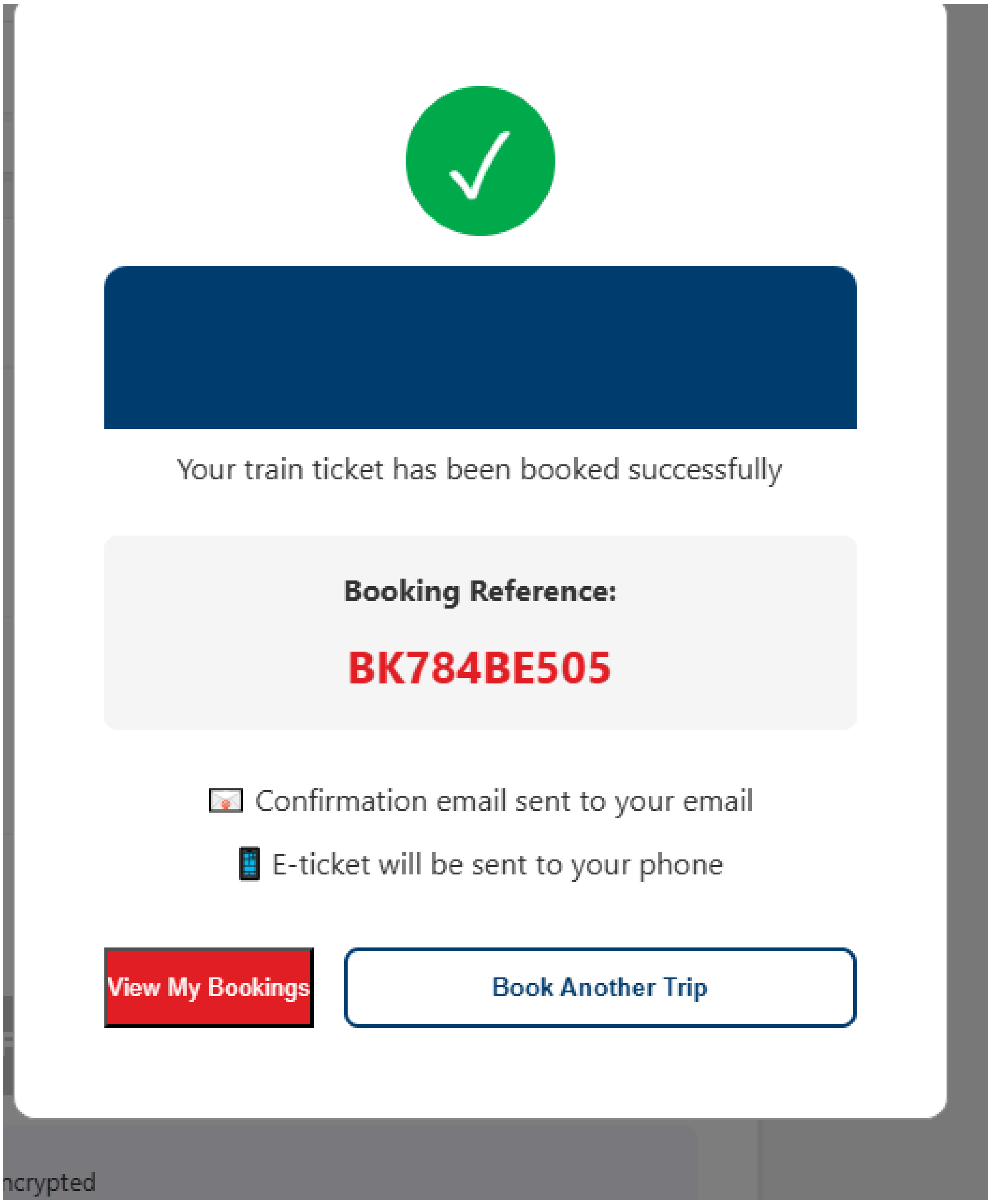
2.8. Confirm Booking

Step 1: After completing all booking information, click the **Confirm & Pay** button.

Step 2: Upon confirmation, a success modal will display:

✓ Booking Successful!

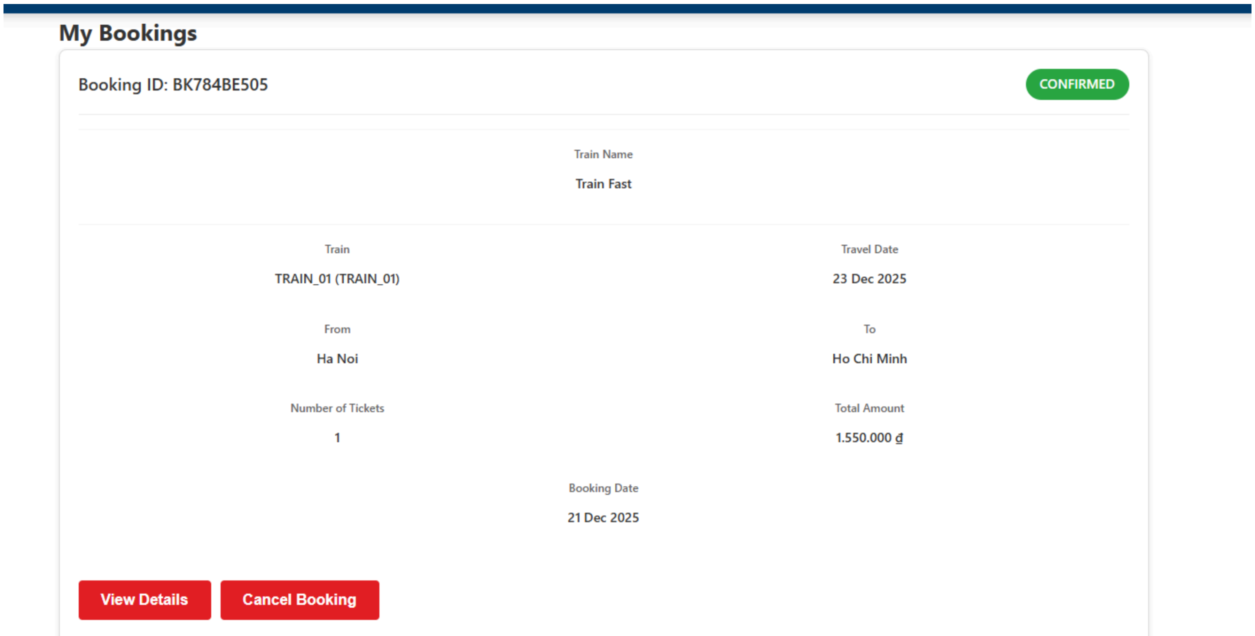
Booking Reference: BK20250115001
Coach: Toa A1
Seat: A5



2.9. View My Bookings

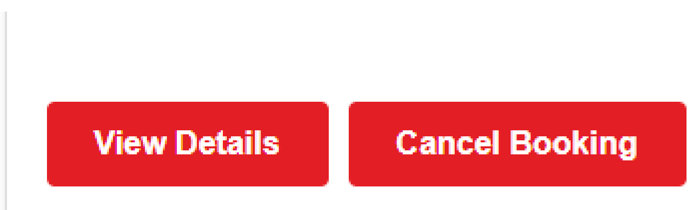
Step 1: After logging in, locate the **My Bookings** option in the navigation menu.

Step 2: Click on **My Bookings** to navigate to the bookings page.



2.10. Cancel Booking

Step 1: From the “My Bookings” page, locate a booking you wish to cancel.



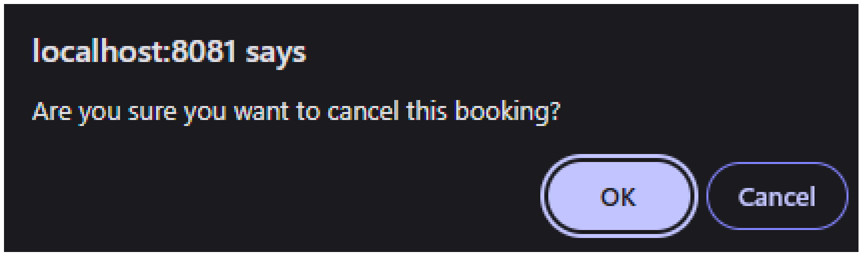
Step 2: Click the **Cancel Booking** button next to the booking.

Planned Cancellation Flow:

1. Verify booking eligibility for cancellation (based on travel date)

2. Show cancellation confirmation dialog

3. Process cancellation (mark booking as cancelled, free up seats)



3. ADMIN ACCESS CONTROL

3.1 Administrator Login & Access

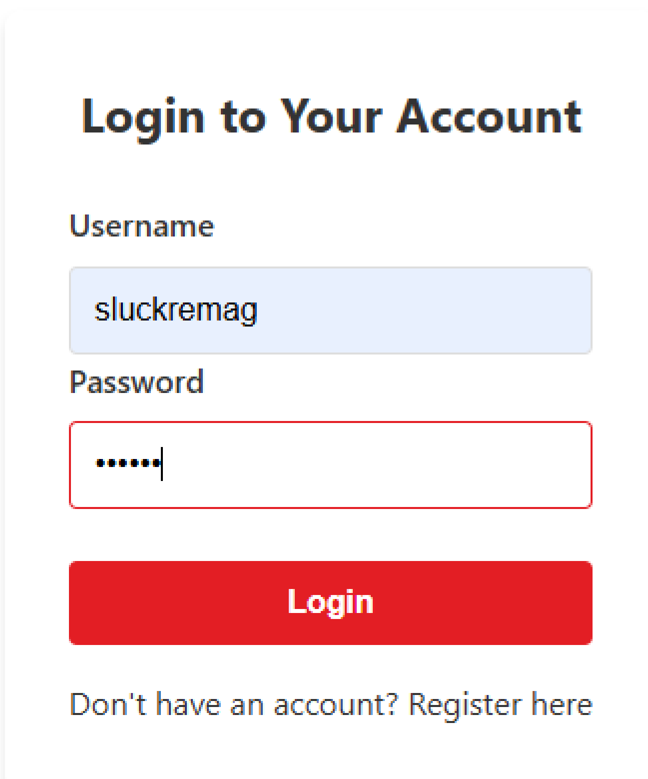
Step 1: Access the Railway Booking System homepage and click the “Login” button.

Step 2: On the login page, enter administrator credentials:

- Username: (ADMIN account username)

- Password: (ADMIN account password)

Note: Only accounts with role = 'ADMIN' in the User table can access administrative functions.



Step 3: After successful login, the navigation bar will display:

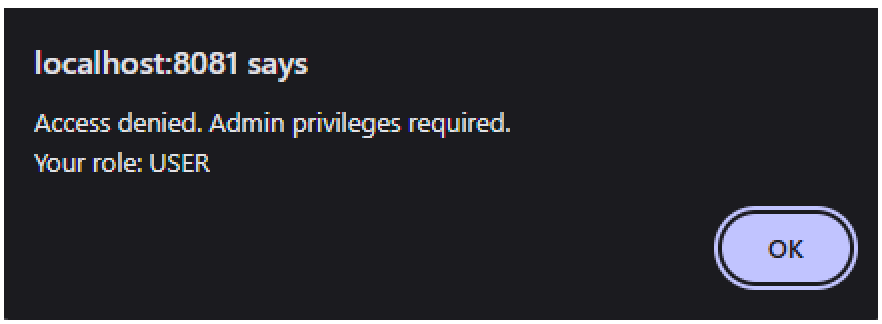
- "Hello, [Admin Username]"

- Additional "Admin Panel" link (visible only to administrators)

- Other standard navigation options

Step 4: Click on the "Admin Panel" link to access the administrative dashboard.

The system will verify your role before granting access. If you are not an administrator, you will be redirected to the homepage with an "Access denied" message.



Role-Based Access Control:

The system implements two-layer protection for the admin panel:

1. **Frontend Protection:** The Admin Panel link is hidden from non-administrator users in the navigation menu.

2. **Access Control:** When accessing admin.html directly, the system checks:

   – User is logged in → If not, redirect to login page

   – User role is 'ADMIN' → If not, redirect to homepage with alert

3.2. Access Admin Dashboard

Step 1: After logging in as an administrator, click on the "Admin Panel" link from the navigation menu.

The admin dashboard URL is:

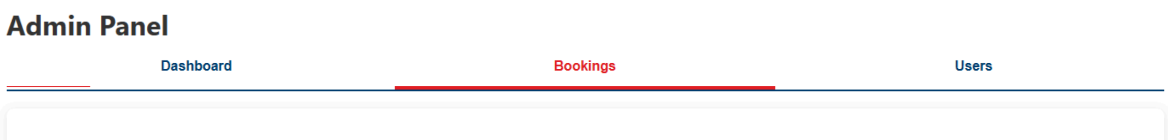
http://localhost:8080/admin.html

Step 2: The Admin Dashboard will load, displaying a tabbed interface with the following sections:

- **Dashboard** - System statistics and overview

- **Users** - User management (view user accounts)

- **Bookings** - Booking management (view, delete bookings)



Step 3: The Dashboard tab displays real-time system statistics:

- **Total Bookings**

- Number of all bookings in the system

- **Total Users** - Number of registered users

- **Total Trains** - Number of trains in the system

- **Cancelled Bookings** - Number of cancelled bookings

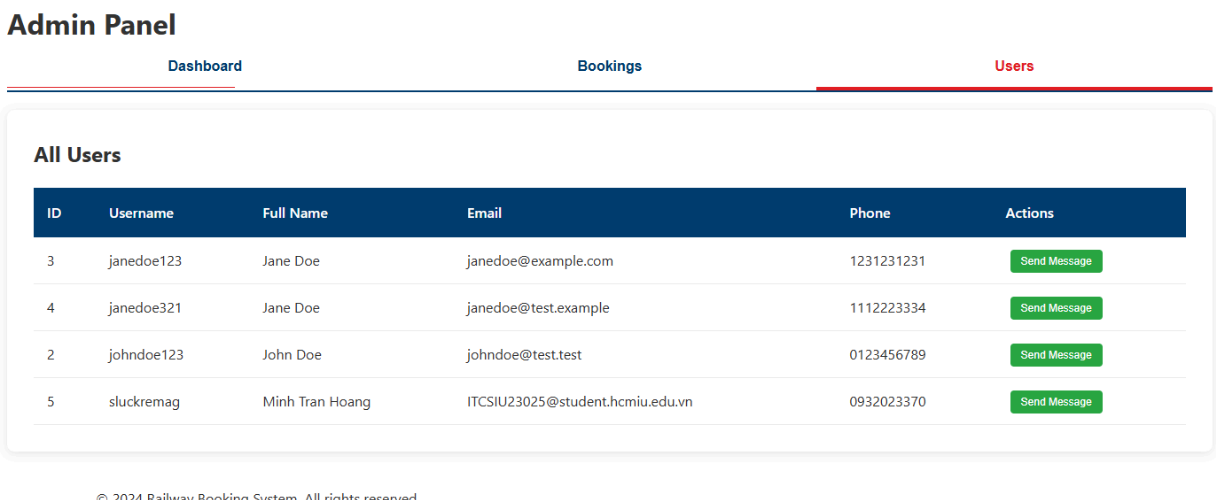
These statistics are fetched from the backend API:

GET /api/admin/stats

Step 4: Navigation between different management sections is achieved by clicking on the respective tabs. Each tab loads data dynamically from the backend API.

3.3. View All Users (UC19)

Step 1: Click on the **"Users"** tab in the admin panel.



ID	Username	Full Name	Email	Phone	Actions
3	janedoe123	Jane Doe	janedoe@example.com	1231231231	Send Message
4	janedoe321	Jane Doe	janedoe@test.example	1112223334	Send Message
2	johndoe123	John Doe	johndoe@test.test	0123456789	Send Message
5	sluckremag	Minh Tran Hoang	ITCSIU23025@student.hcmiu.edu.vn	0932023370	Send Message

Step 2: The system fetches all registered users:

GET /api/admin/users

Step 3: A table displays all users with the following columns: - **User ID** (unique identifier) - **Username** - **Full Name** - **Email** - **Phone** - **Role** (USER or ADMIN)

Note: Password information is never displayed for security reasons.

Step 4: The users list shows both regular users and administrators. Admin accounts are marked with their role.

Sample users:

- User ID: 1, Username: admin, Role: ADMIN

- User ID: 2, Username: john_doe, Role: USER

- User ID: 3, Username: jane_smith, Role: USER

Security Note:

The users table displays sensitive information and should only be accessible to administrators. Passwords are encrypted using BCrypt and are never exposed through the API.

3.4. View All Bookings (UC20)

Step 1: Click on the **“Bookings”** tab in the admin panel.

Step 2: The system fetches all bookings from all users:

GET /api/admin/bookings

Step 3: A table displays all bookings with the following columns:

- **Booking ID** (unique reference)

- **User** (username or user ID)

- **Number of Tickets**

- **Total Amount** (in VND)

- **Status** (CONFIRMED, CANCELLED, etc.)

- **Booking Date** - **Actions** (View Details, Delete buttons)

Step 4: The bookings list shows comprehensive information about all reservations in the system. You can scroll through the list to review all bookings.

Sample bookings:

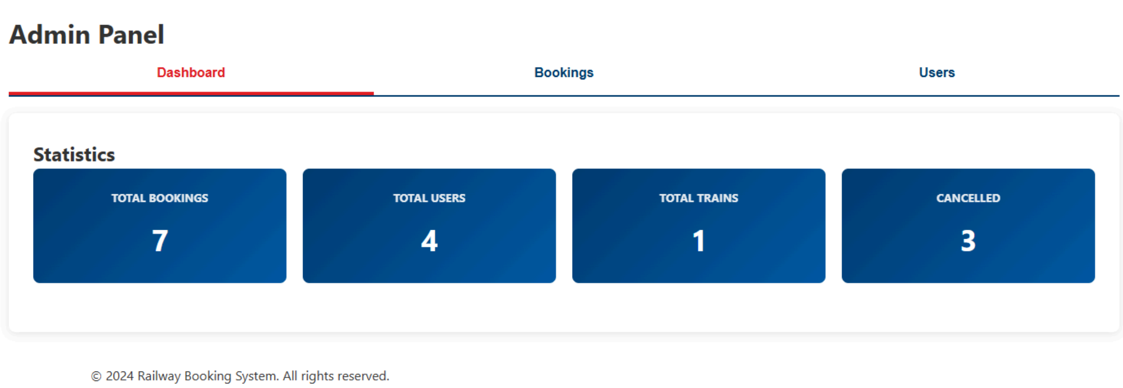
- BK001, User: john_doe, Tickets: 2, Total: 1,700,000 VND, Status: CONFIRMED

- BK002, User: jane_smith, Tickets: 1, Total: 850,000 VND, Status: CONFIRMED

- BK003, User: admin, Tickets: 3, Total: 2,550,000 VND, Status: CANCELLED

3.5. System Statistics Overview

Step 1: On the main **Dashboard** tab, the system displays real-time statistics.



Step 2: The statistics are fetched from the backend:

GET /api/admin/stats

Response:

```
{
  "totalBookings": 45,
  "totalUsers": 23,
  "totalTrains": 4,
  "cancelledBookings": 3
}
```

Step 3: Statistics are displayed in card format:

Step 4: Statistics update automatically when you navigate back to the Dashboard tab, providing real-time system insights.

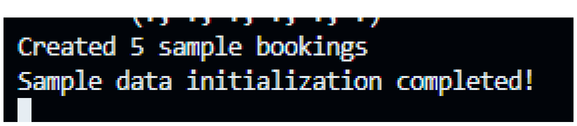
3.6. System Data Seeding

Purpose: The system uses a DataInitializer component to populate the database with sample data on application startup.

Step 1: When the Spring Boot application starts, the DataInitializer component runs automatically.

Implementation:

```
@Component
public class DataInitializer implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
    // Initialize stations, trains, and schedules
    }
}
```



Step 2: The initializer creates sample data:

Stations Created: - ST001 - Hanoi - ST002 - Hai Phong - ST003 - Nam Dinh - ST004 - Thanh Hoa - ST005 - Vinh - ST007 - Hue - ST014 - Ho Chi Minh City - And more…

Step 3: Sample trains are created: - SE1 - Reunification Express SE1 - SE3 - Reunification Express SE3 - SE5 - Reunification Express SE5 - SE7 - Reunification Express SE7

Step 4: Complete train schedules are generated, connecting stations with arrival/departure times for each train route.

Benefits of Data Initialization: - Provides realistic test data for development - Allows immediate testing without manual data entry - Demonstrates system capabilities - Can be disabled in production environments

3.7. Standardized Error Responses

All admin API endpoints return consistent error responses:

Error Response Format:

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Resource not found",
  "details": "Train with ID SE99 not found"
}
```

[IMAGE PLACEHOLDER: Error response structure diagram]

Common Error Codes:

Status Code	Error Type	Description
400	Bad Request	Invalid input or validation failure
404	Not Found	Requested resource does not exist
409	Conflict	Resource already exists (duplicate)
500	Internal Server Error	Server-side error occurred

3.8. Input Validation

Backend Validation (Spring Boot):

Uses @Valid annotation with constraint annotations:

```
@Entity
@Table(name = "schedule", uniqueConstraints = {
        @UniqueConstraint(name = "schedule_train_id_sequence_no_key", columnNames = { "train_id", "sequence_no" })
})
```

Frontend Validation (JavaScript):

```
if (loginForm) {
  loginForm.addEventListener('submit', async function (e) {
    e.preventDefault();

    const username = document.getElementById('username').value.trim();
    const password = document.getElementById('password').value;

    if (!username || !password) {
      showMessage('Please fill in all fields', 'error');
      return;
    }
```

3.9. Role-Based Access Control

Implementation:

Frontend Security:

```
// Check user role before showing admin panel link
 function updateNavigation() {
        const user = getCurrentUser();
        const adminLinks = document.querySelectorAll('a[href="admin.html"]');

        adminLinks.forEach(link => {
        if (user && user.role === 'ADMIN') {
         link.style.display = 'block';
        } else {
         link.style.display = 'none';
        }
        });
 }
```

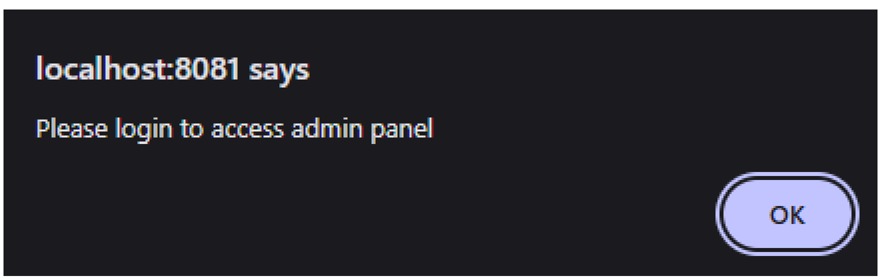
```
const adminLinks = document.querySelectorAll('a[href="admin.html"]');
adminLinks.forEach(link => {
  if (user && user.role === 'ADMIN') {
    link.style.display = 'inline'; k
    console.log('Admin link shown for user:', user.username, 'role:', user.role);
  } else {
    link.style.display = 'none';
    console.log('Admin link hidden. User:', user ? user.username : 'not logged in', 'role:', user ? user.role : 'N/A');
  }
});
```

Page-Level Access Control:

```
// admin.html - Check role on page load
document.addEventListener('DOMContentLoaded', function() {
    const user = getCurrentUser();

    if (!user) {
    alert('Please login to access admin panel');
    window.location.href = 'login.html';
    return;
    }

    if (user.role !== 'ADMIN') {
    alert('Access denied. Admin privileges required.');
    window.location.href = 'index.html';
     return;
    }

    // Load admin data
    loadDashboard();
});
```

```javascript
document.addEventListener('DOMContentLoaded', function() {
    const user = getCurrentUser();
    console.log('Admin panel access check - User:', user);
    console.log('User role:', user ? user.role : 'N/A');
    console.log('Role type:', user ? typeof user.role : 'N/A');
    console.log('Is ADMIN?', user ? user.role === 'ADMIN' : false);

    if (!user) {
        alert('Please login to access admin panel');
        window.location.href = 'login.html';
        return;
    }
    if (user.role !== 'ADMIN') {
        alert('Access denied. Admin privileges required.\nYour role: ' +
        user.role);
        window.location.href = 'index.html';
        return;
    }
    console.log('Access granted to admin panel');
});
```

Step 1: When a non-admin user tries to access the admin panel:



Step 2: The system checks their role and denies access:

The user is redirected to the homepage.

Step 3: Admin users see the admin panel link and can access all features

3.10. Password Security

Password Encryption:

User passwords are encrypted using BCrypt before storing in the database:

```java
@Service
 public class AuthService {
        public User registerUser(UserRequestDTO request) {
        String encryptedPassword = BCrypt.hashpw(
```

```
        request.getPassword(),
        BCrypt.gensalt()
        );

        User user = new User();
        user.setPassword(encryptedPassword);
        // ...
        }
}
```

Password Verification:

During login, passwords are verified using BCrypt:

```
public boolean login(String username, String password) {
        User user = userRepository.findByUsername(username);
        if (user == null) return false;

        return BCrypt.checkpw(password, user.getPassword());
}
```

Security Features:

- Passwords are never stored in plain text

- Passwords are salted and hashed using BCrypt

- Passwords are never exposed through API responses

- Password verification uses constant-time comparison

IV. CONCLUSION

The Railway Management System project demonstrates the practical application of modern web development technologies and Agile methodology. Through careful requirement analysis, structured design, and iterative development, we have created a functional system that addresses real-world train ticket booking challenges.

This project has provided valuable learning experiences in full-stack development, database design, REST API implementation, and user experience design. The modular architecture ensures that the system is maintainable, scalable, and ready for future enhancements such as payment gateway integration, email notifications, and real-time seat availability updates.

V. REFERENCES

Spring Boot — Spring Boot documentation and guides. Available at: https://spring.io/projects/spring-boot

Spring Data JPA — Documentation for JPA repositories and entity mapping. Available at: https://spring.io/projects/spring-data-jpa

PostgreSQL — Official documentation for PostgreSQL database. Available at: https://www.postgresql.org/docs/

BCrypt (Spring Security) — Password hashing using BCrypt. See Spring Security docs: https://spring.io/projects/spring-security

Maven — Build and dependency management. https://maven.apache.org/

MDN Web Docs — Reference for HTML/CSS/JavaScript used in the frontend. https://developer.mozilla.org/